

DevOps Course — Interview Question Bank

Lessons 1–21 · 20 topics · three difficulty tiers per topic · answer keys included

Questions derived from Lessons 1–21 of this repo. Every topic has three parts:

PART	LEVEL	PURPOSE
1. Regular	Junior / warm-up	Does the candidate know the vocabulary and the happy path?
2. Advanced	Mid / senior	Can they reason about <i>why</i> , about trade-offs, and about production?
3. Tricky	Senior / screening filter	Gotchas, counter-intuitive behaviour, and questions where the obvious answer is wrong .

Tricky questions include a ► **Listening for** line — the answer key, so the interviewer knows what separates a real answer from a confident-sounding one.

Several tricky questions are built on real bugs and footguns present in this repo's own lesson code. Those are marked ► **from our code**.

Table of contents

1. DevOps Core Concepts & Releases (L1)	11. AMIs & Launch Templates (L12)
2. SSH, Nginx & Cost Comparison (L2)	12. Autoscaling & Load Balancers (L13)
3. Ansible (L3, L4)	13. Storage: EBS, EFS, S3 (L14)
4. Virtualization vs. Containerization (L5)	14. Route 53 & DNS (L15)
5. Docker Fundamentals (L6)	15. IaC & Terraform Fundamentals (L16, L17)
6. Docker Networking (L7)	16. Terraform Variables & Modules (L18)
7. Multi-stage Builds & Compose (L8)	17. Terraform State & Backends (L19)
8. CI/CD (L9)	18. Kubernetes Architecture (L20)
9. AWS Intro: VPC, EC2, IAM (L10)	19. Kubernetes Manifests (L21)
10. AWS VPC Deep Dive: NAT, Bastion, EIP (L11)	20. Cross-cutting / Scenario Round

1. DevOps Core Concepts & Releases

(Lesson 1 — git tag releases, three-tier apps, frontend deployment, Nginx, VPS/VDS)

Part 1 — Regular

1. What is DevOps? Name three problems it exists to solve.
2. What is a **three-tier application**? Name the tiers and give a concrete example of each.
3. What is a **git tag based release**? Why tag instead of deploying "whatever is on `main`"?
4. Explain **semantic versioning** (`MAJOR.MINOR.PATCH`). Which number moves when you fix a bug? When you break an API?
5. What is **CSR** (client-side rendering) vs **SSR** (server-side rendering)? How does the *deployment artifact* differ between the two?
6. What is a **VPS**? What is a **VDS**? What is the practical difference?
7. Why put **Nginx** in front of an application instead of exposing the app directly?

Part 2 — Advanced

1. For a CSR app: the build output goes to a bucket, served through a CDN. Walk through the full deployment: what gets uploaded, what cache headers each file type gets, and what invalidation you trigger.
2. Compare **blue/green**, **canary**, and **rolling** deployments. For each: what infrastructure does it require, and how fast is rollback?
3. Your release is a git tag. How do you guarantee the artifact built from tag `v1.2.0` is *bit-for-bit* the thing that reaches production three environments later?
4. What is an **annotated** tag vs a **lightweight** tag? Which should a release use and why?
5. Explain the **expand / contract** (parallel change) pattern for a database migration that renames a column, with zero downtime.
6. What are the **four DORA metrics**? Which one is the hardest to game, and why?
7. When is a monolith the correct architecture? Argue against microservices for a 5-person team.

Part 3 — Tricky

1. You deploy a new CSR frontend. Users who had the site open — or who visit within the hour — get a blank page and `ChunkLoadError` in the console. New incognito visitors are fine. What happened?

► **Listening for:** their browser cached the *old* `index.html`, which references old hashed chunk filenames that the deploy deleted from the bucket. Fixes: `index.html` must be served with `no-cache`/short TTL while hashed assets get immutable long TTL; keep N previous builds' assets in the bucket instead of purging; and handle chunk-load failures with a reload. Weak answer: "clear the CDN cache" — that makes it *worse*.

2. `git tag v1.0.0 && git push` — the tag is not on the remote. Why?

► **Listening for:** `git push` does not push tags. Needs `git push origin v1.0.0` or `--tags / --follow-tags`. Follow-up: why is `--follow-tags` safer than `--tags`? (only pushes annotated tags reachable from the pushed commits).

3. A colleague fixes a bad release by force-moving the tag: `git tag -f v1.2.0 && git push -f --tags`. Your CI already built `v1.2.0`. What breaks, and what's the correct move?

► **Listening for:** tags are the immutability contract — the registry now has an image tagged `v1.2.0` built from *different* source, anyone who already fetched keeps the old tag, and provenance is destroyed. Correct move: release `v1.2.1`. Never move a released tag.

4. `git describe` returns "fatal: no tags can describe" even though `git tag` clearly lists tags. Why?

► **Listening for:** `git describe` only considers **annotated** tags by default; theirs are lightweight. `git describe --tags` includes them. Also: the tag must be an *ancestor* of HEAD — a tag on another branch can't describe you.

5. Your CI computes the version as "git tag if present, else short SHA". Two developers merge to `main` within the same minute. Both pipelines push `:latest`. What is now running in production after the deploy job runs?

► **Listening for:** `latest` is a race, not a version. Whichever push landed second wins, which may be the *older* commit if pipelines finish out of order. Deploy must reference the immutable version tag, never `latest`.

6. Is SSR always better for performance than CSR?

► **Listening for:** no — SSR improves first contentful paint and SEO but adds a server you must scale, cache and keep alive; it moves cost from the client to your bill, and hydration can make TTI *worse*. The honest answer names the trade, not a winner.

7. What does "three-tier" have to do with three servers?

► **Listening for:** nothing. Tiers are a *logical* separation (presentation / logic / data). All three can run on one host; one tier can span 50 hosts. Candidates who conflate tier with machine haven't thought about it.

2. SSH, Nginx & Cost Comparison

(Lesson 2)

Part 1 — Regular

1. What happens, step by step, when you run `ssh user@host`?
2. Public key vs password authentication — why do we disable password auth?
3. Where do the keys live on each side? (`~/.ssh/id_*`, `~/.ssh/authorized_keys`)
4. What does `ssh-agent` do?
5. What is a **reverse proxy**? Name four jobs Nginx does in front of an app.
6. In Nginx, what is a `server` block? A `location` block? An `upstream` block?
7. Compare the monthly cost drivers of a VPS vs an equivalent cloud VM. What is *not* in the sticker price on either side?

Part 2 — Advanced

1. Explain `ssh -L`, `ssh -R`, and `ssh -D`. Give one legitimate use for each.
2. What is host key verification protecting you from? What does `StrictHostKeyChecking=no` give up?
3. Configure Nginx as a TLS-terminating reverse proxy for an app on `127.0.0.1:3000`. Which headers must you forward, and why does the app need `X-Forwarded-Proto`?
4. Nginx returns **502** vs **504** — what does each tell you about where the problem is?
5. How do `worker_processes` and `worker_connections` set your theoretical max concurrency? What OS limit will you hit first?
6. What is the difference between `root` and `alias` in a `location` block?
7. Cost comparison: at what point does on-prem/colo beat cloud? Build the argument with concrete cost categories.

Part 3 — Tricky

1. `proxy_pass http://backend;` vs `proxy_pass http://backend/;` inside `location /api/ { }`. A request for `/api/users` reaches the backend as what, in each case?

► **Listening for:** without trailing slash → `/api/users` passed through unchanged. With trailing slash → the matched `location` prefix is *stripped*, backend sees `/users`. This one trailing slash is the single most common Nginx production bug.

2. Your backend container is recreated with a new IP. Nginx keeps 502-ing until you reload it, even though the DNS name is correct. Why?

► **Listening for:** with a static hostname in `proxy_pass`, Nginx resolves DNS **once at startup** and caches it for the process lifetime. Fix: define a `resolver` and use a *variable* in `proxy_pass` (`set $up http://backend;` `proxy_pass $up;`) to force runtime resolution — or use the commercial `resolve` parameter on `upstream server`.

3. You have `location /images { }` (prefix), `location ~ /\.png$ { }` (regex), and `location ^~ /images { }`

} . A request for `/images/logo.png` — which block wins?

► **Listening for:** order of precedence, not file order: exact `=` → then longest `^~` prefix (which *stops* regex evaluation) → then regexes in file order → then longest plain prefix. So `^~ /images` wins here.

4. Your app logs every client IP as `172.18.0.1`. You add `X-Forwarded-For` handling and now you're logging attacker-controlled values. What did you get wrong?

► **Listening for:** `X-Forwarded-For` is client-supplied and forgeable. You must *append* at the trusted proxy and only trust the hops you control — `set_real_ip_from <trusted CIDR> + real_ip_header` in Nginx, and never trust the leftmost entry blindly.

5. `chmod 777 ~/.ssh/authorized_keys` "to make sure it's readable" — now key auth fails entirely. Why?

► **Listening for:** `sshd` refuses group/world-writable `authorized_keys` (and a writable `~` or `~/.ssh`) as a hijack risk — `StrictModes`. Needs `700` on `~/.ssh`, `600` on the file. The error only appears in the *server's* auth log, which is the second half of the answer.

6. A developer runs `ssh -R 9000:localhost:22 bastion` from a laptop inside your network. Why should your security team care more about this than about any inbound rule?

► **Listening for:** reverse tunnel = an outbound connection that creates an *inbound* path, punching straight through the firewall's asymmetry. Egress filtering and `AllowTcpForwarding no` / `GatewayPorts no` on the bastion are the controls. Great senior-level security instinct check.

7. Which is cheaper for a workload that runs 24/7 at steady load: cloud or a VPS? Now the workload is 2 hours a day. Did your answer change, and what does that tell you about what cloud actually sells?

► **Listening for:** cloud sells *elasticity and speed of provisioning*, not cheap compute. Steady 24/7 load is where cloud loses on raw price (hence reserved instances/savings plans); bursty or unknown load is where it wins. Anyone answering "cloud is cheaper" flat has never seen a bill.

3. Ansible

(Lessons 3, 4 — inventory, ad-hoc, playbooks, roles, templates, facts)

Part 1 — Regular

1. What is Ansible and what does **agentless** mean? What must exist on a managed node?
2. What is an **inventory**? Show a group with group-level vars.
3. What is **idempotency** and why does it matter for configuration management?
4. **Ad-hoc command vs playbook** — when do you reach for each?
5. What is a **module**? Name five you've used and what each does.
6. What is a **role**? What are the standard directories inside one?
7. What are **facts** and which module gathers them?
8. What do the flags `-b`, `-K`, `--check`, `--diff`, `--limit` do?

Part 2 — Advanced

1. `command` vs `shell` — what's the actual difference, and why is `command` preferred? Neither is idempotent by default: how do you *make* a `command` task idempotent?
2. **Handlers** — when do they run? What is `meta: flush_handlers` for and when do you need it?
3. Walk through Ansible **variable precedence** from lowest to highest. Where do `role/defaults` sit and where does `--extra-vars` sit?
4. `apt: state=present` vs `state=latest` — which is idempotent, which is reproducible, and why can't you have both?
5. How do you do a **rolling** restart across 30 web servers with no full outage? (`serial`, `max_fail_percentage`, `any_errors_fatal`)
6. How do you manage secrets in Ansible? Compare **Ansible Vault** against an external secret manager.
7. What is a **template** (Jinja2) vs a **copy**? Why is `template` + handler the standard config-file pattern?
8. `forks` vs `serial` — what does each control?

Part 3 — Tricky

1. **FROM OUR CODE** Our inventory sets:

```
ansible_ssh_common_args=-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
```

This is in a lesson repo. What exactly have we disabled, what attack does it enable, and what should production use instead?

► **Listening for:** we've turned off host-key verification *and* thrown away the `known_hosts` record, so every connection blindly trusts whatever answers on that IP — a textbook MITM / IP-reuse hijack, and it silently keeps working after an attacker takes the address. Production: pre-seed `known_hosts` (`ssh-keyscan` at provision time, or SSH certificates / `@cert-authority`), and set `host_key_checking = True`. Acceptable *only* for ephemeral throwaway lab VMs — and the candidate should say so out loud.

2. **FROM OUR CODE** This repo has two playbooks against hosts: `web: nginx-playbook.yaml` ends with `service: state=started enabled=true`; `simple-nginx-playbook.yaml` ends with `service: state=stopped enabled=false`. Both are "idempotent". What is the state of nginx on those hosts?

► **Listening for:** idempotency is per-run convergence toward a *declared* state — it says nothing about two playbooks declaring **contradictory** states. The answer is "whichever ran last", which means the real state is a function of run order, i.e. undefined. This is configuration drift caused by the config management tool itself. The fix is one source of truth per resource.

3. **FROM OUR CODE** `nginx-playbook.yaml` installs nginx, starts it, then does:

```
- template:
  src: index.html.j2
  dest: /var/www/html/index.html
```

Name two independent problems with this task.

► **Listening for:** (a) `index.html.j2` doesn't exist in the repo → the task fails at runtime; (b) even if it existed, a `template` task that changes a *config* file needs to `notify` a handler to reload/restart the service, otherwise the change is on disk but not live. Bonus: the task has no `name:`, so output is unreadable; and `web_port: 8080` is only ever `debug`-printed, never actually applied to nginx's config — the var is decorative.

4. `--check` mode reports "ok" for your whole playbook, so you run it for real and it fails on task 3. How can a dry run be wrong?

► **Listening for:** `--check` is a *prediction*, not a simulation. `command/shell` tasks are skipped (so anything depending on their registered output is evaluated against nothing), tasks with `check_mode: no` actually run, and any task whose behaviour depends on a change an *earlier* skipped task would have made will report garbage. Modules also vary in how faithfully they support check mode.

5. A task shows `changed` on every single run even though nothing on the host differs. Give three distinct causes.

► **Listening for:** (a) `command/shell` with no `creates/removes/changed_when` — always changed by definition; (b) a `template` whose output is non-deterministic (timestamp, dict ordering, `ansible_date_time`); (c) `state: latest`, or a `lineinfile` regexp that doesn't match the line it writes so it appends every time; (d) `file` with a `mode/owner` the system immediately rewrites. Chronic false-`changed` destroys the signal value of `changed`.

6. **FROM OUR CODE** Our inventory pins `ansible_python_interpreter=/usr/bin/python3`. Why would you ever need to set that by hand, and what breaks if you set it wrong?

► **Listening for:** Ansible ships Python modules to the target and executes them there; interpreter discovery guesses, and guesses badly on distros with multiple Pythons or a bare-bones image. Wrong value → `/usr/bin/python: not found`, or worse: a Python without the module's dependencies (e.g. `python3-apt` missing → the `apt` module fails confusingly). Pinning it is correct practice, not a hack.

7. Your playbook has `become: yes` at play level. One task must run as the `postgres` user, another must run

as the *local* control node. How?

► **Listening for:** `become_user: postgres` on that task (still via `become`), and `delegate_to: localhost` (+ `become: no`) for the local one — with the note that `local_action/connection: local` is the older form, and that `delegate_to: localhost` still runs once per host in the play unless you add `run_once: true`.

8. Handlers notified in a play that fails halfway — do they run?

► **Listening for:** by default **no** — handlers only flush at the end of the play (or at a `flush_handlers`), so a mid-play failure means a service was reconfigured on disk but never restarted: the host is left in an inconsistent state. `-force-handlers` changes this. This is a genuinely nasty production surprise.

4. Virtualization vs. Containerization

(Lesson 5 — hypervisors, overcommit, isolation, migration, leader election)

Part 1 — Regular

1. VM vs container — explain the difference in one diagram (what sits where in the stack).
2. What is a **hypervisor**? Type 1 vs Type 2, with an example of each.
3. Why can a container start in milliseconds while a VM takes tens of seconds?
4. Which is more **isolated**, and why?
5. What does **overcommit** mean?
6. What is **swap** and why do we not want it on a production database host?
7. **Stateful vs stateless** application — give an example of each and say which is easier to scale.
8. What is **live (online) migration** vs **offline migration**?

Part 2 — Advanced

1. Why is **CPU overcommit** generally acceptable on a hypervisor while **RAM overcommit** is not? What is the queueing/scheduling mechanic behind CPU overcommit?
2. What is the "golden space" you must leave for the hypervisor/donor host, and what happens when you don't?
3. Why is **storage overcommit** (thin provisioning) risky for VMs? Describe the failure mode when the datastore hits 100%.
4. What must be true for **live migration** to work? (shared storage or storage migration, same CPU feature set / EVC, network reachability, memory pre-copy + final stop-and-copy)
5. Explain **leader / follower** election. Why an **odd** number of voting members?
6. What is **split brain**? Give a concrete data-loss scenario, and name two mechanisms that prevent it (quorum, fencing/STONITH, lease + generation number).
7. When would you deliberately choose a VM over a container in 2026?
8. What does gVisor / Kata Containers / Firecracker exist to solve?

Part 3 — Tricky

1. A container exceeds its CPU limit. A container exceeds its memory limit. Describe what happens in each case — they are not the same kind of event.

► **Listening for:** CPU is a *compressible* resource → the cgroup CFS quota **throttles** the process (it just runs slower; check `nr_throttled`). Memory is *incompressible* → the kernel **OOM-kills** the process inside the cgroup (exit 137). Candidates who say "requests get denied" or "the container is killed" for CPU have memorized the wrong model. Follow-up: which of these is invisible in `docker stats` and shows up only as latency? (throttling)

2. You run `nproc` and `free -m` inside a container limited to 0.5 CPU and 256 MiB. What do they report?

► **Listening for:** the **host's** values — `/proc` is not namespaced for CPU count or memory. Consequence: any runtime that sizes its thread pool or heap from "available memory/CPU" (JVM before container awareness, Node's libuv pool, Go's `GOMAXPROCS`) will over-allocate and get OOM-killed or throttled to death. Fixes: `GOMAXPROCS` / `XX:MaxRAMPercentage`, `lxcfs`, or explicit limits in the app config. This is the single best question in this section.

3. "Containers are isolated." Your container runs as root and you mounted `-v /:/host`. Who owns the machine?

► **Listening for:** the attacker. Root in a container is (by default, without user namespaces) root on the host — same UID 0, and the parent OS can reach into the container's filesystem while a compromised privileged container can reach out. Mitigations: user namespace remapping, non-root `USER`, dropped capabilities, read-only rootfs, seccomp/AppArmor. Contrast with a VM, where the guest kernel boundary means an escape requires a hypervisor vulnerability.

4. A 5-node etcd/consensus cluster network-partitions 3 / 2. Which side keeps serving writes? Now a 4-node cluster partitions 2 / 2 — which side wins?

► **Listening for:** quorum is $\text{floor}(N/2)+1$. 5 nodes → 3 needed → the 3-side serves, the 2-side goes read-only/unavailable. 4 nodes → 3 needed → **neither** side has quorum, the whole cluster stops accepting writes. That's the point of odd numbers: 4 nodes tolerate the same single failure as 3 while adding a partition mode where *everyone* loses. Adding a node can make you *less* available.

5. Your hypervisor host has 128 GB RAM and 20 VMs summing to 200 GB of allocated RAM. Everything is fine for months, then one Tuesday all 20 VMs become unusably slow at once, with no config change. What happened?

► **Listening for:** RAM overcommit was masked by unused (never-touched) guest pages plus ballooning/page sharing. Once real working sets grew past physical RAM, the host started swapping *guest memory* to disk — every guest's memory access became a disk I/O. The failure is nonlinear and simultaneous, which is exactly why RAM overcommit is unacceptable in production while CPU overcommit is survivable.

6. Live migration completes "successfully" but the application drops every in-flight TCP connection. What was misconfigured?

► **Listening for:** live migration preserves memory state, but the VM must keep its **IP and L2 adjacency** — it has to land in the same broadcast domain/VLAN, and the network has to learn the new port (RARP/GARP announcement). Cross-subnet migration without an overlay changes the IP and kills every socket. Also: the final stop-and-copy pause must be shorter than app/TCP timeouts, which fails for very write-heavy guests.

7. Is a container a "lightweight VM"?

► **Listening for:** no — a container is just a **process on the host kernel** with namespaces (what it can see: pid, net, mnt, uts, ipc, user) and cgroups (what it can use: cpu, memory, io, pids). There is no guest kernel, no virtual hardware, no boot. This is why you can't run a Windows container on a Linux kernel, why a host kernel panic takes all containers with it, and why kernel-level CVEs are shared blast radius.

5. Docker Fundamentals

(Lesson 6 — chroot, jails, cgroups, OCI, CLI↔daemon, images, layers, Dockerfiles)

Part 1 — Regular

1. What is **chroot** and why is it *not* enough to be a container?
2. What do **namespaces** give a container? Name four and what each hides.
3. What do **cgroups** give a container?
4. What is the **OCI** and what problem did it solve? Name and describe its three specs (**Image**, **Runtime**, **Distribution**).
5. Explain **Docker CLI** → **dockerd** → **containerd** → **runc**. Who does what?
6. **Image vs container** — define both in one sentence each.
7. What is a **layer**? Why does Dockerfile instruction *order* affect build caching?
8. Difference between **CMD** and **ENTRYPOINT**? Between **COPY** and **ADD**? Between **EXPOSE** and **-p**?

Part 2 — Advanced

1. What is the **union filesystem** / copy-on-write layer? What happens when a running container writes to a file that exists in a lower read-only layer?
2. Trace what `docker run -d nginx` does end to end, naming every component involved.
3. What is an **image digest** vs a **tag**? Which one should a production deployment pin, and why?
4. `RUN apt-get update && apt-get install -y curl` on one line vs two separate **RUN** lines — explain the caching bug in the two-line version.
5. What does a **HEALTHCHECK** actually do? Who consumes the result?
6. How do you run a container as **non-root** properly? What has to change besides adding **USER**?
7. `docker system df` — explain "reclaimable" space. Where does disk actually go on a busy build host?
8. What is `--init` / PID 1 in a container? What is the zombie-reaping and signal-forwarding problem?

Part 3 — Tricky

1. **FROM OUR CODE** Our homework asks for a **HEALTHCHECK** hitting `/health`, and our images are `node:20-alpine`. A student writes:

```
HEALTHCHECK CMD curl -f http://localhost:3000/health || exit 1
```

The container reports **unhealthy** forever, but the app works fine from the host. Why — and then: does Docker restart the unhealthy container?

► **Listening for:** two separate gotchas. (a) `curl` is **not installed** in alpine — busybox provides `wget`, so use `wget -q0- ... || exit 1` or install `curl`. (b) Plain `docker run` does **nothing** with an unhealthy status — it only reports it. Only an orchestrator acts on it: Swarm reschedules, Compose's `depends_on: condition: service_healthy` gates startup, ALB/K8s use their own probes. "It will auto-restart" is wrong and very commonly believed.

2. Your Dockerfile copies `package*.json`, runs `npm ci`, then `COPY . .`. You add a `.dockerignore` with `node_modules`. A colleague asks why that matters — "we already installed inside the image". What do you tell them?

► **Listening for:** without it, `COPY . .` overwrites the image's `node_modules` with the host's, which was built for a different OS/libc/CPU (macOS arm64 → linux/amd64) — native modules break at runtime with cryptic errors. It also invalidates the cache on every host `npm install` and bloats the build context (slow `docker build` even for a one-line change). Bonus: `.dockerignore` also keeps `.git`, `.env`, and keys out of the image and out of the build context sent to the daemon.

3. You add `USER node` at the end of your Dockerfile. The app now crashes with `EACCES: permission denied, open '/app/dist/...'` — but only sometimes, and only when it writes. Explain, and fix it without `chmod 777`.

► **Listening for:** every `COPY` before `USER` lands owned by `root` with default perms; the app can read but not write, and any directory it must create files in is root-owned. Fix: `COPY --chown=node:node`, or `RUN chown -R node:node /app` before `USER`, or better — make the app not write into its own code directory (writable paths should be a volume or `/tmp`). Also worth hearing: after `USER`, subsequent `RUN`s lose root, so ordering matters.

4. Two `docker build` runs on two machines from the same commit produce images with different digests. Is the build broken?

► **Listening for:** no — Docker builds are not reproducible by default: timestamps in layer metadata, `apt-get`/`npm` pulling different upstream versions behind a floating tag, `FROM node:20-alpine` resolving to a new base image, build-arg and buildkit differences. That's why you pin base images by `digest`, use lockfiles, and treat the built artifact (pushed once) as the unit of promotion rather than rebuilding per environment. Mentioning `SOURCE_DATE_EPOCH` / reproducible-build efforts is a bonus.

5. You `docker build` on your M-series Mac, push, and the image crash-loops on your EC2 instance with `exec format error`. What happened, and give two fixes.

► **Listening for:** you built `linux/arm64`, the target is `linux/amd64`. Fixes: `docker build --platform linux/amd64` (slow, via emulation), `docker buildx build --platform linux/amd64,linux/arm64 --push` for a multi-arch manifest list, or build in CI on the target arch. Follow-up: what is a manifest list and how does the runtime pick from it?

6. `docker run` a container, `rm -rf /` some critical file inside it, then `docker restart` it. Is the file back?

► **Listening for:** no. `restart` reuses the *same* container and its writable layer, so the damage persists. Only removing the container and running a new one from the image gives you a clean writable layer. This is the image-vs-container distinction with teeth — and it's why "have you tried restarting it" behaves differently in Docker than people expect.

7. `docker exec` into a container and `kill 1`. Then explain why your app didn't shut down gracefully on `docker stop` either.

► **Listening for:** PID 1 in a namespace has special signal semantics — it ignores signals it has no handler for, so a shell-form `CMD` (which wraps in `/bin/sh -c`) means `sh` is PID 1, does **not** forward `SIGTERM` to your app, and after the 10s grace period `docker stop` sends `SIGKILL` — no graceful drain. Fixes: exec-form `CMD ["node", "server.js"]`, `--init/tini` for reaping, and an actual `SIGTERM` handler in the app.

8. Why does the Docker CLI being a *client* to a daemon matter for security? What does adding a user to the `docker` group actually grant?

► **Listening for:** root. The daemon runs as root and the socket is an unauthenticated root API — anyone who can reach `/var/run/docker.sock` can `docker run -v /:/host --privileged` and own the host, no sudo needed. Hence: `docker` group $\approx$ passwordless root, mounting the socket into a container (common in CI!) is a full escape, and this is a core motivation for rootless mode / Podman.

6. Docker Networking

(Lesson 7 — bridge, host, overlay, macvlan, ipvlan, none, DNS, publishing)

Part 1 — Regular

1. Name the six network drivers from the lesson and give one sentence + one use case for each: `bridge`, `host`, `overlay`, `macvlan`, `ipvlan`, `none`.
2. What is the **default bridge** (`docker0`) vs a **user-defined bridge**?
3. On a user-defined bridge, how does `web` resolve `api`? Why doesn't that work on the default bridge?
4. What does `-p 8080:3000` actually change? What happens if you don't publish?
5. What does `EXPOSE 3000` in a Dockerfile do on its own?
6. What is `--network none` for?
7. How do you inspect a network's subnet and connected containers?

Part 2 — Advanced

1. What actually implements port publishing? (`iptables/nftables` DNAT in the `DOCKER` chain + `docker-proxy`) — what does that mean for your carefully written `ufw` rules?
2. How does an **overlay** network carry traffic between hosts? (VXLAN encapsulation, control plane, why MTU matters)
3. **macvlan vs ipvlan** — what's the difference at L2/L3, and when do you need each?
4. Why do cloud providers (AWS) generally block `macvlan`?
5. Design network-level isolation for a three-tier app so that `frontend` cannot reach `db` at all. Then explain what that does *not* protect against.
6. `--network host` — what do you gain and what do you lose? Why is it common for monitoring/network agents?
7. What is Docker's embedded DNS server (`127.0.0.11`)? What happens to a container's DNS if you override `--dns`?
8. How do you reach a service on the *host* from inside a container, portably?

Part 3 — Tricky

1. `web` and `api` are on the same user-defined bridge. `api` was started with `-p 8080:3000`. From inside `web`, `curl http://api:8080/health` fails but `curl http://api:3000/health` works. Explain precisely.

► **Listening for:** published ports are a **host-namespaces** DNAT rule; they do not exist inside the container network. Container-to-container traffic goes to the container's *own* port (3000) directly over the bridge. `-p` is purely for ingress from outside. Corollary: you should **not** publish internal services at all — publishing is an attack surface, not a connectivity requirement.

2. **FROM OUR CODE** Our homework says "on macOS/Windows host networking behaves differently — mention that." So: what does `--network host` do on Docker Desktop for Mac, and why?

► **Listening for:** containers on Mac/Windows don't run on your OS — they run in a **Linux VM**. `--network host` means the VM's host network, not macOS's, so your service is not on `localhost` of the Mac and `-p` is what actually bridges the VM to macOS. (Newer Docker Desktop has an opt-in host-networking feature with caveats.) The general lesson: "localhost" is namespace-relative, and on Mac there's an extra hop people forget.

3. A container on a user-defined bridge can resolve `api`. You then attach it to a second network. `getent hosts api` now sometimes returns a different IP. What's going on?

► **Listening for:** a container has one interface (and one address) **per network** it's attached to; Docker's embedded DNS returns the address on a shared network, and with multiple attachments resolution/ordering can differ per network. Practical consequences: don't assume one container = one IP, be explicit about which network a connection should use, and remember that a multi-homed "gateway" container is exactly how you *bypass* your tiering isolation.

4. You wrote `ufw` rules to block port 8080 from the internet. `docker run -p 8080:80` is reachable from the internet anyway. Why, and what do you do?

► **Listening for:** Docker inserts its DNAT rules in the `nat/DOCKER` chain, which is traversed **before** the `filter INPUT` chain that `ufw` manages — so `ufw` never sees the packet as INPUT. Fixes: publish to a specific interface (`-p 127.0.0.1:8080:80`), use the `DOCKER-USER` chain (which Docker guarantees is evaluated first and doesn't overwrite), set `iptables: false` and manage it yourself, or put the security boundary in the cloud security group instead. This bites real production hosts constantly.

5. `--network none` gives you no connectivity. Can the container still talk to anything at all? And name a real use for it beyond "an exercise".

► **Listening for:** it gets only a loopback interface — so intra-container `localhost` works, and it can still touch mounted volumes, IPC if shared, and any file descriptor passed in. Real uses: untrusted batch computation on data mounted in, running a build/lint step with a guarantee of no network access (supply-chain hardening / hermetic builds), forensic analysis of a malware sample.

6. Two containers on the *same* user-defined bridge. You want them to be unable to talk to each other. `docker network create` gives you a flag for that — and it has a catch. What is it?

► **Listening for:** `--internal` blocks *external* access, not container-to-container; the one they want is `com.docker.network.bridge.enable_icc=false` (inter-container communication off) — and the catch is that with ICC disabled, published-port/DNAT paths and links behave differently, and you lose the DNS-based service discovery pattern that made user-defined bridges attractive. The clean answer is usually **separate networks**, which is what our Lesson 8 topology does.

7. Your overlay network works for HTTP but large TLS handshakes and file uploads hang. Nothing is in the logs. What do you check first?

► **Listening for:** MTU. VXLAN adds ~50 bytes of encapsulation; if the underlay MTU is 1500 and the container MTU is also 1500, full-size packets fragment or get silently dropped, and ICMP "fragmentation needed" is often blocked → **PMTU black hole**. Symptom is exactly this: small requests fine, large ones hang. Fix: lower the container/overlay MTU (e.g. 1450) or enable jumbo frames on the underlay. Textbook senior-level debugging

answer.

7. Multi-stage Builds & Compose

(Lesson 8 — builder/runtime stages, prod dep pruning, Compose, tiered networks, named volumes)

Part 1 — Regular

1. Why use a **multi-stage build**? What ends up in the final image vs. what stays in the builder?
2. What does `npm ci --omit=dev` do? Why is it right in the runtime stage but wrong in the builder?
3. `npm ci` vs `npm install` — which belongs in a Dockerfile and why?
4. What does **Compose** give you over a pile of `docker run` commands?
5. What is a **named volume**? Why can't a database keep its data in the container's writable layer?
6. `depends_on` — what does it guarantee, and what does it **not** guarantee?
7. In our `docker-compose.yml`, why is only `frontend` published to the host?
8. `docker compose down` vs `docker compose down -v` — what's the difference and why does it matter?

Part 2 — Advanced

1. Walk through our two-stage recipe (`task.md`) and explain what each stage's cache boundary buys you on a code-only change.
2. `COPY --from=builder /app/dist ./dist` — how does BuildKit know it can skip the builder stage entirely in some builds?
3. How do you make `backend` wait for `db` to be genuinely **ready**, not just started? Give two approaches and their trade-offs.
4. In our topology (`front-tier: frontend+backend`, `back-tier: backend+db`), `backend` is multi-homed. What does that mean for your security model?
5. Named volume vs bind mount vs `tmpfs` — pick one for: postgres data, local development source, and secrets. Justify each.
6. How do you get secrets into Compose services without baking them into the image or committing them?
7. What is a `.env` file in Compose, and what is the difference between Compose variable substitution and container environment variables? (a classic confusion)
8. Your final image is still 900 MB after multi-staging. List the things you'd check.

Part 3 — Tricky

1. Your builder stage is **FROM node:20 (Debian)** and your runner is **FROM node:20-alpine**. Everything builds. The container crashes at startup with an error about a `.node` file. Why?

► **Listening for:** native modules compiled against `glibc` in the builder cannot load under `musl` in alpine. `npm ci` in the runner would rebuild them correctly — but if you copied `node_modules` across stages, you copied `glibc` binaries. Fixes: identical base OS in both stages, or `node:20-alpine` in both, or install prod deps *in* the runner stage (which is exactly what our `task.md` recipe does — and this question tests whether they know *why* the recipe copies `package*.json` again rather than copying `node_modules`).

2. **FROM OUR CODE** Our `docker-compose.yml` has `db` on `back-tier` only and `frontend` on `front-tier` only, so `frontend` can't reach `db`. A student "fixes a connection issue" by adding `back-tier` to `frontend`. Nothing else changes. What did they just do?

► **Listening for:** they deleted the entire security boundary the exercise was about — `frontend` (the only internet-exposed service, i.e. the most likely thing to be compromised) can now reach the database directly. Also worth hearing: network isolation here is *defense in depth*, not authentication — `backend` reaching `db` with `POSTGRES_PASSWORD: app` over a shared bridge is still weak, and the real fix for the original connection issue was almost certainly a wrong `DB_HOST` or a readiness race.

3. **FROM OUR CODE** Our compose has `depends_on: - db` on `backend`. The stack works on your machine and fails in CI with "connection refused" roughly one run in three. Explain, and give the actual fix.

► **Listening for:** `depends_on` (short syntax) only orders **container start**, not readiness — postgres accepts TCP only after `initdb` finishes, which on a cold CI volume is seconds. Your machine "works" because the DB image is cached and warm. Fixes, best to worst: `depends_on: db: condition: service_healthy` with a `healthcheck` using `pg_isready`; a wait-for-it/dockerize wrapper; **or the correct production answer** — the app retries its DB connection with backoff, because in a real orchestrator nothing guarantees dependency ordering at all.

4. `docker compose down` then `up -d` and your data is still there. Then someone runs `docker compose down -v` and it's gone. Later you `docker compose up -d` and postgres starts fine on an empty database — no error. Why is that the dangerous part?

► **Listening for:** the postgres entrypoint only runs `initdb` when the data directory is **empty**, so a destroyed volume looks identical to a first-time install — the stack comes up green, healthchecks pass, and the failure is *silent data loss*, discovered later by users. Mitigations: never rely on local volumes for anything you care about, back up + `restore-test`, and treat "the DB came up empty" as a paging alert rather than a normal startup path.

5. Your `docker compose up --build` rebuilds and redeploys `backend` after you edit source. But an environment-variable change in `docker-compose.yml` sometimes doesn't take effect. What's the rule?

► **Listening for:** Compose recreates a container when its *config hash* changes, but `up` without `--force-recreate` can reuse a container whose spec it considers unchanged, and — crucially — an `.env` / shell variable change that Compose substitutes is picked up, while a change baked into the **image** (an `ENV` in the Dockerfile) needs a rebuild, and a change to an already-running process's env needs a **new container** (you cannot mutate a running container's environment). Candidates should land on: environment changes require container replacement, not restart.

6. You add `healthcheck` to every service as our bonus asks. Your CI pipeline now takes 4 minutes longer and one service flaps between healthy and unhealthy. What did you get wrong about healthcheck parameters?

► **Listening for:** `interval`, `timeout`, `retries`, and especially `start_period` — without a start period, a slow-booting service fails its first N checks and gets marked unhealthy; with `retries` too high you add `interval × retries` to every startup. Flapping usually means the check itself is too heavy (hitting a route that touches the DB) or the timeout is shorter than the service's p99. Good answer distinguishes **liveness** (is it broken?) from **readiness** (can it take traffic?) — a healthcheck that queries downstream dependencies turns one slow dependency into a cascading outage.

7. Multi-stage build, and you need a private-repo SSH key to `npm ci` in the builder. You `COPY id_rsa` in the builder stage, use it, `RUN rm id_rsa`, and never copy it to the runner. Is the key in the final image? Is it recoverable?

► **Listening for:** not in the *final* image (the builder stage is discarded), which is genuinely better than a single-stage build where `rm` in a later layer leaves the key in the earlier layer forever. **But:** the key is still in the local build cache and in the builder stage's layers, so anyone with the cache or a pushed intermediate/`--target` build has it, and it appears in build logs/history. Correct answer: **BuildKit secrets** (`RUN --mount=type=secret`) or `--mount=type=ssh`, which never materialize in any layer.

8. CI/CD

(Lesson 9 — CI vs CD, artifacts, versioning, registries, lint/test/build/deploy, Sonar)

Part 1 — Regular

1. Define **Continuous Integration**, **Continuous Delivery**, and **Continuous Deployment**. What separates the last two?
2. What is a **build**? What is an **artifact**? For our Node app, what is the artifact and what should it contain?
3. What does "**build once, deploy the same thing everywhere**" mean and why does it matter?
4. Why version artifacts? Explain `APP_VERSION = git tag, else short SHA`.
5. What's the point of the **lint** stage vs the **test** stage? Why lint first?
6. What is **SonarQube** and what is a **quality gate**? How is it different from a linter?
7. What is an **artifact registry** (Artifactory, GitLab Container Registry)? Why not just rebuild the image in each stage?
8. In `.gitlab-ci.yml`: what are `stages`, `script`, `before_script`, `artifacts`, `rules`, `services`?

Part 2 — Advanced

1. Why does the `build` job need `docker:dind` as a `services:` entry? What is `DOCKER_TLS_CERTDIR: "/certs"` doing?
2. What's the security problem with Docker-in-Docker in CI, and what are the alternatives? (`--privileged` requirement, socket mounting, Kaniko/Buildah/BuildKit rootless)
3. Our `test` job publishes `reports: junit:`. What does GitLab do with that, and why is a JUnit report better than reading the log?
4. Where should a **quality gate** block in `lint → test → sonar → build → deploy`? Argue for your placement.
5. Why is `deploy when: manual` and limited to `main`? When would you make it automatic, and what must exist first?
6. How do you cache `node_modules` across jobs safely? What is the cache **key** and why does `npm ci` complicate this?
7. How do you handle secrets in a pipeline? What are masked vs protected variables, and what is OIDC / short-lived credentials replacing?
8. Design a pipeline that promotes one artifact through `dev → staging → prod`. What changes between environments if the artifact can't?

Part 3 — Tricky

1. **FROM OUR CODE** Our `.gitlab-ci.yml` has:

```
variables:
  APP_VERSION: "${CI_COMMIT_TAG:-$CI_COMMIT_SHORT_SHA}"
```

Does this do what it looks like it does?

► **Listening for:** No. GitLab's `variables:` block does simple variable *expansion*, not shell parameter expansion — `${VAR:-default}` is not supported there. On a branch pipeline `CI_COMMIT_TAG` is empty, so `APP_VERSION` becomes a literal mangled string (and gets used as a **Docker tag**, which may fail or produce a garbage tag). Correct approaches: compute it in `script:/before_script:` where a real shell runs (`APP_VERSION="${CI_COMMIT_TAG:-$CI_COMMIT_SHORT_SHA}"`), or use `workflow:/rules:` with `variables:` per case, or `!reference/extends` for tag vs branch pipelines. This is the best question in the section because the file *looks* obviously correct.

2. ► **FROM OUR CODE** The `deploy` job is:

```
rules:
  - if: '$CI_COMMIT_BRANCH == "main"'
    when: manual
```

Your release process is "tag a commit to release". Walk me through releasing `v1.2.0`.

► **Listening for:** you *can't*. On a tag pipeline `CI_COMMIT_BRANCH` is **empty** (`CI_COMMIT_TAG` is set instead), so the rule doesn't match and the `deploy` job **doesn't exist** in the pipeline. The versioning story (tag → image tag) and the deploy gate contradict each other. Fix: `- if: '$CI_COMMIT_TAG'` (plus optionally the main-branch rule), which is also the moment to ask whether deploy should trigger on tags at all vs. an environment-promotion job.

3. ► **FROM OUR CODE** Our `lint` job sets `image: node:20-alpine`. Our `test` job sets no `image:` at all but runs `npm ci`. What happens?

► **Listening for:** it falls back to the runner's **default image** (whatever the runner config or instance default is — on shared runners often a Ruby/Debian image). `npm/node` may be missing or a wildly different version than `lint` and `build` used, so you can lint and build with Node 20 while testing on whatever the runner felt like. Symptoms: works today, breaks when the runner is updated, and version-specific bugs escape. Fix: pin `image:` on every job (or set a top-level `default: image:`), and pin the same version everywhere including the Dockerfile.

4. ► **FROM OUR CODE** `build` pushes both `$IMAGE:$APP_VERSION` and `$IMAGE:latest` — and the job runs on **every** branch. What is in `:latest` right now?

► **Listening for:** whatever branch pushed most recently — a half-finished feature branch, quite possibly. If any environment or `docker compose` pulls `latest`, you've deployed unreviewed code. Fixes: only tag `latest` on default-branch or tag pipelines (`rules:`), or drop `latest` entirely and always deploy immutable versions. Bonus: mutable tags also break rollback and audit, and interact badly with `imagePullPolicy: IfNotPresent`.

5. Your pipeline is green. `npm test` passes. Production breaks on the exact code path the tests cover. Give three ways a green pipeline lies.

► **Listening for:** (a) the tests don't assert what they appear to (no assertion / mocked the system under test / a test that passes vacuously when the fixture is empty); (b) the pipeline built and tested different artifacts than it deployed (rebuild per stage, `latest` drift, config differing per environment); (c) test environment ≠ prod

(different Node/OS/libc, different DB version, no real network latency, no concurrency); (d) the failing path is *config* or *data*, which unit tests never see. Senior answer also mentions: "a green pipeline is evidence, not proof" and that this is why you need staged rollout + observability, not just more tests.

6. A junior adds `allow_failure: true` to the flaky `test` job so pipelines stop blocking. Defend or reject, and what would you do instead?

► **Listening for:** reject — you haven't fixed flakiness, you've deleted your quality gate while keeping the appearance of one, which is worse than having no test stage because it manufactures false confidence. Real options: quarantine the specific flaky test (tagged, tracked, with an owner and a deadline), retry with `retry: max: 2` only for infrastructure-class failures, and fix the root cause (shared state, time/timezone dependence, ordering, real network calls, missing awaits).

7. Why is caching `node_modules` between jobs sometimes *slower* and sometimes *wrong* than just running `npm ci`?

► **Listening for:** slower — the cache is archived, uploaded, downloaded, and extracted; for a small dep tree on a fast registry/mirror that costs more than the install, and cache upload happens even when unused. Wrong — `npm ci` exists precisely to produce an exact, clean tree from the lockfile; restoring a stale `node_modules` on top defeats that (partial trees, wrong platform binaries from a differently-configured runner, no invalidation if your key isn't the lockfile hash). Correct pattern: key the cache on `package-lock.json`'s hash, cache the `npm cache dir` (`~/.npm`) rather than `node_modules`, and keep `npm ci`.

8. Where exactly does a Sonar quality gate belong, and what does "quality gate blocks the pipeline" require that a linter does not?

► **Listening for:** after tests (it wants coverage reports as input), before build/deploy. Unlike a linter — which runs locally, exits non-zero itself, and is deterministic on the diff — a Sonar gate is **asynchronous and server-side**: the scanner uploads, the server computes, and you must explicitly *wait* for the gate result (`sonar.qualitygate.wait=true`) or your job passes before the verdict exists. Also: gates are usually scoped to **new code** ("clean as you code") rather than the whole codebase, otherwise a legacy repo can never go green.

9. AWS Intro: VPC, EC2, IAM

(Lesson 10)

Part 1 — Regular

1. What is cloud computing? Give three concrete reasons to use it over on-prem.
2. **IaaS / PaaS / SaaS** — define each with one AWS example. **Public / private / hybrid** cloud?
3. What is a **VPC**? What is a **subnet**? What makes a subnet **public** vs **private** — be precise.
4. What is **EC2**? What is an **AMI**, an **instance type**, a **key pair**?
5. **IAM**: define user, group, role, policy. What's the difference between an *identity-based* and a *resource-based* policy?
6. Is a **security group** the same as a firewall? Compare SG (stateful) with **NACL** (stateless).
7. Where does an **Internet Gateway** fit? What does attaching one actually enable?
8. **Stopping** vs **terminating** an EC2 instance — what happens to the root EBS volume in each case?

Part 2 — Advanced

1. Plan CIDRs for a 3-AZ VPC with public, private, and database tiers. Why not just use `10.0.0.0/16` as one flat subnet? What can't you change later?
2. How does IAM policy evaluation work? Walk through explicit deny, explicit allow, implicit deny, and where SCPs / permission boundaries / session policies fit.
3. Why is an **IAM role** with `AssumeRole` better than an access key on an instance? What is IMDS and what is IMDSv2 protecting against?
4. What is a **VPC endpoint** (gateway vs interface)? What problem does it solve for a private subnet talking to S3?
5. Security group **referencing another security group** as source — how does that work under the hood, and why is it better than a CIDR?
6. Explain the difference between the **main** route table and an explicit association. What happens to a subnet with no explicit association?
7. Design least-privilege IAM for a CI pipeline that must push to ECR and update one ECS service. What does the policy look like?
8. Why can leaving an unused **Elastic IP** or IGW cost money? What else silently bills after you "delete everything"?

Part 3 — Tricky

1. You created a VPC with an IGW attached, a route `0.0.0.0/0 → igw` in a route table, an instance with a public IP, and an SG allowing 22 from your IP. SSH times out. Name five remaining causes.

► **Listening for:** (a) the route table isn't **associated** with that subnet (traffic is using the main route table); (b) a **NACL** blocking inbound 22 or — subtler — blocking **outbound ephemeral ports** 1024–65535 (NACLs are stateless, so replies need an explicit rule); (c) no public IP actually assigned (`map_public_ip_on_launch` false and no EIP); (d) OS-level firewall (`ufw`/`iptables`) or `sshd` not running / instance failed to boot; (e) wrong username for the AMI (`ec2-user` vs `ubuntu`) or wrong/mismatched key pair; (f) your "own IP" changed, or you're behind

CGNAT/VPN so your real source IP isn't what you whitelisted. A senior candidate distinguishes **timeout** (network/SG/NACL/route) from **connection refused** (reached the host, nothing listening) from **permission denied** (network fine, auth wrong) — that triage is the real answer.

2. Your security group allows inbound 443 and you have deleted *all* egress rules. Does an HTTPS request from the internet get a response?

► **Listening for:** yes. Security groups are **stateful** — return traffic for an allowed inbound connection is permitted regardless of egress rules. (What breaks is traffic the *instance* initiates, e.g. `yum update`.) Contrast: a NACL is stateless, so the same setup would drop the response. This single question separates people who memorized "SG = stateful" from people who understand it.

3. "A subnet is public because you ticked auto-assign public IP" Correct this, then answer: an instance in that subnet has a public IP and the SG allows all — can it reach the internet?

► **Listening for:** public/private is a property of the **route table** — a `0.0.0.0/0` route to an IGW. Auto-assign public IP without an IGW route gives you an instance with a public address and no path: unreachable in both directions. Conversely a route to an IGW without a public IP on the instance is also dead — the IGW performs 1:1 NAT and needs a public address to map. **Both** are required.

4. You're told "S3 is global, so my bucket is region-less." True?

► **Listening for:** the **bucket namespace** is global (names are globally unique, which is why `my-bucket` is taken), but every bucket **lives in exactly one region** and the data doesn't leave it unless you replicate. The correct global/regional/zonal taxonomy: IAM & Route 53 = global; VPC, route tables, IGW, S3 buckets, EIP = regional; **subnets, EBS volumes, EC2 instances = zonal**. The tricky ones are S3 (global name, regional data) and EIP (regional allocation, but only usable in that region).

5. Your teammate says "us-east-1a is a specific datacenter, so let's both deploy to us-east-1a for low latency between accounts." What's wrong?

► **Listening for:** AZ **names** are randomized per account — your `us-east-1a` and their `us-east-1a` are (probably) different physical zones. The stable identifier is the **AZ ID** (`use1-az1`), exposed via `describe-availability-zones`. AWS did this deliberately so customers don't all pile into "a". This matters for cross-account VPC peering/PrivateLink placement and for cross-AZ data transfer charges.

6. An IAM user is in a group with **AdministratorAccess**. They also have an inline policy denying **s3:DeleteObject**. Can they delete an S3 object? Now the bucket policy explicitly allows them to. Can they?

► **Listening for:** no, and still no. **Explicit deny always wins** — it cannot be overridden by any allow, in any policy type, at any level (identity, resource, SCP, boundary). The evaluation order is: explicit deny → SCP/boundary must allow → explicit allow → otherwise implicit deny. Follow-up worth asking: what if the deny is in an SCP and the user is the account root? (still denied — SCPs apply to root in member accounts).

7. You delete an EC2 instance and the VPC still won't delete, complaining about dependencies. What's holding it, and what's the correct teardown order?

► **Listening for:** ENIs (from the instance, or left behind by NAT gateways, load balancers, RDS, VPC endpoints, Lambda-in-VPC), plus the IGW attachment and non-default SGs/route tables/NACLs. Order: instances/ENI-owning services (LB, NAT GW, RDS, endpoints) → **release EIPs** → route table associations → route tables → detach +

delete IGW → subnets → SGs → VPC. The senior point: "the console let me delete the instance" ≠ "the money stopped" — NAT gateways and EIPs are the classic surprise bill.

10. AWS VPC Deep Dive: NAT, Bastion, EIP

(Lesson 11 — zonal/regional/global, AZ, NAT GW vs NAT instance, virtual router, EIP, CGNAT, bastion, iptables vs ufw)

Part 1 — Regular

1. Classify as global / regional / zonal: IAM, S3, VPC, subnet, EC2 instance, IGW, EIP, Route 53, EBS volume.
2. **Region vs Availability Zone** — what is an AZ physically? Why deploy across several?
3. What is the VPC **virtual router**? How does a subnet get connected to a route table?
4. What problem does a **NAT gateway** solve? Which subnet must it live in, and which route table references it?
5. Why does a private instance need NAT for **outbound** internet but stay unreachable **inbound**?
6. What is an **Elastic IP**? When does it cost you money?
7. What is a **bastion** / **jump host** and why is it better than a public IP on every instance?
8. `iptables` vs `ufw` — what's the relationship?

Part 2 — Advanced

1. **NAT gateway vs NAT instance** — give two pros and two cons of each. What must you disable on a NAT instance, and what `iptables` rule makes it work?
2. What is **CGNAT** and how does it relate to Elastic IPs and to "allow SSH from my IP only"?
3. Why does a NAT gateway have to be in a *public* subnet? Trace a packet from a private instance to the internet and back, naming every hop.
4. High availability for NAT: what breaks when your single NAT gateway's AZ fails? What's the production pattern, and what does it cost?
5. Compare `ProxyJump` vs SSH **agent forwarding** for reaching a private host through a bastion. Which is safer and why?
6. Beyond a bastion: what are the modern alternatives, and what do they buy you? (SSM Session Manager, Instance Connect Endpoint, Tailscale/WireGuard, zero public ingress at all)
7. How would you audit "which of my private instances can reach the internet, and through what?" (route tables, VPC Reachability Analyzer, Flow Logs)
8. When would you use a **NACL** rather than a security group? Give a case SGs genuinely cannot cover.

Part 3 — Tricky

1. You place the NAT gateway in the *private* subnet (it's for private instances, after all) and add `0.0.0.0/0 → natgw` to the private route table. What happens?

► **Listening for:** nothing works, and the failure is a **routing loop** — the NAT GW's own outbound traffic follows the private route table, which points at the NAT GW. A NAT GW must sit in a subnet whose route table points `0.0.0.0/0` at the **IGW**, because NAT translates to its EIP and then needs the IGW to actually egress. Follow-up gold: what happens if you accidentally put `0.0.0.0/0 → natgw` in the *public* route table? (same loop, plus you break the bastion.)

2. A private instance reaches the internet through NAT. Someone port-scans the NAT gateway's Elastic IP.

Can they reach the private instance?

► **Listening for:** no — NAT is **connection-oriented one-way**: translations exist only for flows the private instance *initiated*, and there is no port-forwarding/inbound mapping on a NAT gateway (unlike a NAT instance where you could add DNAT rules). Unsolicited inbound has no translation entry and is dropped. This is *why* "outbound yes, inbound no" is a property of NAT itself, not of a security group.

3. NAT gateway lives in **az-a**. Your private instances are spread across **az-a** and **az-b**, all using one private route table pointing at that NAT GW. Two questions: what breaks if **az-a** fails, and what is it costing you right now even when nothing is broken?

► **Listening for:** (a) **az-b** instances lose **all** outbound internet even though **az-b** is perfectly healthy — you've coupled an AZ-independent workload to a single AZ. Fix: one NAT GW per AZ + one private route table per AZ. (b) Right now, every byte from **az-b** to the internet crosses an AZ boundary and pays **cross-AZ data transfer** on top of NAT GW per-GB processing — so the single-NAT "cost saving" is partly illusory. Bonus: for S3/DynamoDB traffic, a **gateway VPC endpoint** removes NAT charges entirely.

4. **iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE** on your NAT instance, source/dest check disabled, route table updated. Private instances still can't reach the internet. What did you forget?

► **Listening for:** **net.ipv4.ip_forward=1** (and persisting it in **sysctl.conf**) — the box won't forward packets between interfaces at all by default. Also possible: the NAT instance's **security group** must *allow inbound* traffic from the private subnet CIDR (it's a hop, not a destination), FORWARD chain policy is DROP, and the instance needs a public IP/EIP with an IGW route. And the reason source/dest check must be off: EC2 drops packets whose source IP isn't the instance's own — which is exactly what forwarding does.

5. You lock the bastion's SG to "SSH from my IP only". It works from the office, fails from home, and one day a stranger's connection is allowed by your rule. Explain all three.

► **Listening for:** **CGNAT** — your ISP shares one public IPv4 among many subscribers, so (a) your "IP" is really your ISP's NAT pool address, (b) it changes, and (c) whitelisting it whitelists **every other customer behind that same CGNAT address**. Home is a different pool. Conclusion: source-IP allowlists are a weak control for residential users; real answers are a VPN/bastion behind SSO, SSM Session Manager (no inbound at all), or short-lived certificates.

6. Your teammate uses **ssh -A** (agent forwarding) through the bastion because it's convenient. The bastion is shared by 12 people. What's the risk, and what's the alternative?

► **Listening for:** agent forwarding exposes your agent **socket** on the bastion; anyone with root there (or another user who can reach **/tmp/ssh-***) can use your agent to authenticate **as you** to anything your key opens — the key isn't copied, but its *authority* is borrowed for the session's duration, silently. Alternative: **ProxyJump / -J** (or **ProxyCommand**), which keeps all authentication on your laptop and treats the bastion as a dumb TCP relay. If you must forward, use **ssh-add -c** for confirmation and per-host **ForwardAgent** scoping — never **ForwardAgent yes** globally.

7. **ufw allow 22** on your bastion, and it works. Then Docker starts on the same host and publishes a port; your **ufw deny** rules stop applying to it. Explain the relationship between **ufw**, **iptables**, and **nftables** well enough to say why.

► **Listening for:** **ufw** is a *frontend* that generates rules in netfilter (via **iptables/iptables-nft**; **nftables** is the modern backend, **iptables** is often a shim over it). It manages its own chains hooked into **INPUT**. Docker writes **DNAT rules in the nat table's DOCKER chain**, which is traversed before **filter/INPUT** sees the packet as locally

destined — so ufw's INPUT rules never evaluate it. They're not two firewalls; they're two writers to one kernel ruleset with different hook points. Fix: the `DOCKER-USER` chain, or bind-publish to localhost, or enforce at the security group.

8. "IAM is global, so I can use one IAM role in any region." True. "Elastic IP is regional, so I can move it to another region." Which of these is wrong, and what's the general rule you'd give a junior?

► **Listening for:** the second. An EIP is **allocated in and usable only within one region** — you can't move it (you'd need a new EIP, or BYOIP for real address portability). The general rule: scope tells you *where the resource exists*, not *what it can reach*. Global resources (IAM, Route 53, CloudFront, S3 names, WAF for CloudFront) are one control plane; regional resources are duplicated per region; zonal resources (subnet, EBS, instance) pin you to one datacenter and are the ones you must consciously spread for HA.

11. AMIs & Launch Templates

(Lesson 12)

Part 1 — Regular

1. What is an **AMI**? What's in it?
2. What is a **launch template**? What can it specify?
3. Launch template vs the older **launch configuration** — why did templates replace them?
4. What is **user data**? When does it run?
5. What is **cloud-init**?
6. How do you create a custom AMI from a running instance?
7. What is a **golden image** and why would you bake one?

Part 2 — Advanced

1. **Baking (Packer) vs bootstrapping (user data)** — compare on boot time, reproducibility, patch velocity, and debuggability. What's the hybrid everyone actually uses?
2. Launch template **versions** — explain `$Latest` vs `$Default` and why an ASG's choice between them is a deployment decision.
3. How do you roll a new AMI out to an existing ASG? What is an **instance refresh** and what do `MinHealthyPercentage` / warm-up control?
4. AMIs are regional. How do you distribute one image to 6 regions, and what breaks if it's encrypted?
5. How do you find "the latest Amazon Linux 2023 AMI" programmatically? Why is that better than hardcoding an ID?
6. What is a **warm pool**, and what problem does it solve that a smaller AMI wouldn't?
7. Where should application config come from: baked into the AMI, user data, or fetched at runtime? Defend your answer.

Part 3 — Tricky

1. You edit user data on a stopped instance, start it, and your changes don't run. Then you *terminate and relaunch* from the same launch template and they do. Explain.

► **Listening for:** cloud-init runs most modules **once per instance** (`instance-id` scoped, state in `/var/lib/cloud/instances/`), so a reboot/stop-start doesn't re-execute the user-data script. A new instance = new instance-id = fresh run. If you genuinely want per-boot behaviour, use `cloud-init per-boot` scripts (`#cloud-config` with `bootcmd`, or `/var/lib/cloud/scripts/per-boot/`), or clear the cloud-init state (hacky). Practical corollary: user data is **not** a config-management channel; treat it as bootstrap-only.

2. ► **FROM OUR CODE** Two of our Terraform lessons set `ami = "resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"`. What does this do, and name a serious operational downside.

► **Listening for:** it's an EC2 **SSM parameter reference** resolved by the `RunInstances` API at launch, so you always get the current published AMI without a data source. Downside: it's **non-deterministic and invisible to terraform plan** — two applies weeks apart silently produce instances from different images, your

infrastructure isn't reproducible, and you can't roll back to "the AMI that worked". Terraform also can't detect the drift because the attribute string never changes. Production pattern: resolve the AMI explicitly (`data "aws_ssm_parameter" / data "aws_ami"`), pin it in a variable, and bump it as a reviewed change.

3. You update your launch template to a new AMI version. What happens to (a) the 4 instances currently running in the ASG, (b) the next instance the ASG launches for a scale-out event?

► **Listening for:** (a) **nothing** — existing instances are never modified; the launch template is a *blueprint applied at launch*. (b) depends entirely on whether the ASG references `$Latest`, `$Default`, or a pinned version number. So a "deploy" that only updates the template produces a fleet with **mixed versions** — some old, some new, depending on scaling activity — which is a genuinely nasty state to debug. Deliberate rollout requires an **instance refresh** (or a blue/green ASG swap).

4. You bake a golden AMI from a running production instance. New instances launch, join the load balancer, and behave strangely — duplicate host identity, SSH host key warnings, and one service refusing to start. Why?

► **Listening for:** the image captured **machine-specific state**: SSH host keys (now identical across the fleet — a real security problem and the source of the warnings), cloud-init state (so user data won't run), machine-id, logs, cached DHCP/hostname, application state/PID files, and possibly credentials or an agent already registered under the source instance's identity. Correct baking involves a cleanup/sysprep step (`cloud-init clean`, remove host keys, clear logs and machine-id) — which is exactly the sort of thing Packer provisioners are for.

5. Your `data "aws_ami" { most_recent = true }` filter is how you "always get the latest patched image". A colleague calls this dangerous. Who's right?

► **Listening for:** both, and the resolution is the interesting part. It's good for patch velocity and bad for reproducibility — `terraform plan` will show an instance **replacement** the moment upstream publishes a new AMI, meaning an unrelated apply can suddenly destroy and recreate instances, and it can happen mid-incident. Also: `most_recent` with a loose `name` filter can match an image you didn't intend, and you must pin `owners` or you're trusting a name pattern from anyone. Mature pattern: resolve latest in CI, write it to a variable/tfvars as an explicit reviewed bump, and keep `lifecycle { ignore_changes = [ami] }` or an instance-refresh workflow for controlled rollout.

6. An AMI copied to another region launches fine. The same AMI shared with another account fails to launch with a KMS error. What's the missing piece?

► **Listening for:** encrypted snapshots are encrypted with a **KMS key**, and (a) the default AWS-managed EBS key **cannot be shared** across accounts — you must re-encrypt with a **customer-managed key**, and (b) the target account needs `kms:DescribeKey/Decrypt/CreateGrant` on that CMK *and* the snapshot must be shared with it. Cross-region copy needs a CMK in the **destination** region. This trips up nearly every multi-account image pipeline once.

12. Autoscaling & Load Balancers

(Lesson 13)

Part 1 — Regular

1. What is horizontal vs vertical scaling? Which does an ASG do?
2. What is an **Auto Scaling Group**? Explain min / max / **desired capacity**.
3. **ALB vs NLB vs CLB** — which OSI layer, and what's a use case for each?
4. What is a **target group**? What is a **listener**? What is a **rule**?
5. What is a **health check**? What does the LB do when one fails?
6. What is a **scaling policy**? Compare target tracking, step scaling, and scheduled scaling.
7. What is a **cooldown** period and why does it exist?

Part 2 — Advanced

1. Why must an ALB have subnets in at least two AZs? What is **cross-zone load balancing**, and what is the default for ALB vs NLB?
2. Explain **connection draining** / **deregistration delay**. What breaks if it's 0? What breaks if it's 600s?
3. Design autoscaling for a queue worker. Why is CPU the wrong metric, and what's right?
4. **Sticky sessions** — how does an ALB implement them, and what do they cost you in an autoscaling world?
5. What is the difference between an **ELB health check** and an **ASG health check**? What is `HealthCheckGracePeriod` and what does an ASG do with `ELB` health check type?
6. How do you deploy new code to an ASG-backed service with no dropped requests? Compare instance refresh, blue/green target-group swap, and a rolling CodeDeploy strategy.
7. An NLB **preserves the client source IP**. What does that mean for your instances' security groups, and how does it differ from ALB?
8. What is **thundering herd** on scale-out, and how do warm pools / pre-scaling / lifecycle hooks help?

Part 3 — Tricky

1. **ASG desired=3. You manually terminate one instance in the console. Then you manually launch a 4th instance yourself in the same subnet. What does the ASG do in each case?**

► **Listening for:** it replaces the terminated one (desired is a *declared* state it continuously reconciles — same mental model as Terraform or a K8s Deployment). Your manually-launched instance is **invisible** to the ASG — it isn't a member, so it won't be counted, health-checked, or terminated by the ASG, but it *will* keep serving traffic if you registered it with the target group, and it will never be replaced when it dies. This "pet inside a cattle system" is a classic source of an instance nobody can account for.

2. **Your app takes 90 seconds to warm up. You set an ALB health check with interval 10s, threshold 2, and ASG health check type `ELB`. Describe the failure mode.**

► **Listening for:** an infinite **launch–kill loop**: the instance fails health checks at ~20s, the ASG (using ELB health) marks it unhealthy and terminates it, launches a replacement, repeat — burning money and never converging, while `desired` is never met so the incident looks like a capacity problem. Fix: `HealthCheckGracePeriod` > warm-

up time, plus a health endpoint that reports readiness honestly. Follow-up: why is grace period *per instance* rather than per ASG, and what happens if the app's warm-up time grows past it after a code change?

3. You have an NLB in front of instances whose security group allows `0.0.0.0/0` on 443. Works fine. You switch to an ALB and now want to lock the SG down to "only the load balancer". How do you do it for each type, and why are the answers different?

► **Listening for:** ALB has **security groups** of its own, so the instance SG source is the *ALB's security group ID* — clean and IP-independent. NLB (classically) has no SG and **preserves the client source IP**, so the instance SG must allow the actual **client** CIDRs (or the VPC/NLB subnet CIDRs when target type is `instance` vs `ip` — behaviour differs, and NLB security-group support was added later as an opt-in you must enable at creation). The general lesson: source-IP preservation and SG-based filtering pull in opposite directions.

4. A single instance in your ALB target group is serving 80% of traffic; the other five are nearly idle. All health checks pass. Give two independent explanations.

► **Listening for:** (a) **cross-zone load balancing disabled** (NLB default off) with uneven instance distribution across AZs — each AZ's LB node splits evenly among *its* targets, so 1 instance alone in an AZ receiving half the DNS resolutions gets hammered; (b) **sticky sessions** plus long-lived connections: the LB balances *connections*, not requests, so a client that opened one keep-alive connection and then sends 10k requests pins them all to one target — very common with gRPC/HTTP2 and with a small number of high-volume clients. Also acceptable: DNS caching of one LB node's IP by a badly-behaved client.

5. Target tracking on average CPU at 50%. Traffic doubles in 30 seconds. Your service still browns out. Why isn't autoscaling "the answer" here?

► **Listening for:** autoscaling is a **feedback loop with dead time** — CloudWatch metric granularity + alarm evaluation + launch + boot + warm-up + health-check + registration easily totals minutes, and CPU is a *lagging* indicator that saturates before it signals. You cannot scale faster than you can boot. Real answers: over-provision headroom, warm pools / pre-baked AMIs to cut boot time, scale on a **leading** metric (request rate, queue depth, `RequestCountPerTarget`), scheduled scaling for known spikes, and load-shedding/backpressure so degradation is graceful rather than a brownout. Senior candidates say the quiet part: some spikes must be absorbed, not scaled into.

6. Scale-in happens. Users report being logged out and one background job silently disappeared. Both are the same root cause. What is it, and what are the fixes?

► **Listening for:** the terminated instance held **local state** — in-memory sessions (hence logouts, made worse by sticky sessions) and an in-flight job with no external checkpoint. Fixes: externalize session state (Redis/DynamoDB or signed cookies), make workers idempotent with a queue that redelivers on visibility-timeout expiry, use **ASG lifecycle hooks** + `Terminating:Wait` to drain gracefully, honor `SIGTERM`, and set deregistration delay to cover in-flight requests. The framing to listen for: "scale-in is a *planned* failure, so anything that can't survive it can't survive a real one either."

7. Why does an ALB scale by getting *more IP addresses*, and why does that matter for a client that resolved its DNS name an hour ago?

► **Listening for:** an ALB is a fleet of nodes behind a DNS name with a **short TTL (60s)**; scaling out adds nodes and changes the A records. A client (or JVM, or a naive HTTP library) that caches the DNS result indefinitely keeps hammering an IP that may be **removed**, causing connection failures that look like the LB is broken. Consequences: never hardcode an ALB IP, don't whitelist ALB IPs (use its SG or PrivateLink), respect DNS TTL in clients, and if you need a static IP, that's an NLB (one static/EIP per AZ) — which is a big reason NLBs still exist.

13. Storage: EBS, EFS, S3

(Lesson 14)

Part 1 — Regular

1. **Block vs file vs object** storage — define each and map EBS / EFS / S3 onto them.
2. What is an **EBS volume**? Which scope is it (global/regional/zonal)?
3. Compare `gp3`, `io2`, `st1`, `sc1` — what's each for?
4. What is an **EBS snapshot**? Where does it live?
5. What is **EFS** and what protocol does it speak? What is a **mount target**?
6. What is **S3**? What is a bucket, a key, an object? Is there such a thing as a folder in S3?
7. Name the S3 storage classes and when you'd use each.
8. What is **versioning** on a bucket, and what is a **lifecycle policy**?

Part 2 — Advanced

1. Why can't you attach one EBS volume to instances in two AZs? How *do* you move a volume between AZs, and between regions?
2. What is **EBS multi-attach** and what does it *not* give you? (hint: it doesn't make ext4 safe)
3. **EFS throughput modes** and burst credits — describe the failure mode of a burst-mode filesystem under sustained load.
4. When is EFS the wrong choice and EBS right, and vice versa? Consider latency, concurrency, and cost per GB.
5. **S3 consistency** — what are the guarantees today, and what were they before December 2020? Why does that history still show up in old code?
6. How do you make an S3 object publicly readable, and name every layer that can block you.
7. Explain S3 **pre-signed URLs**. Why are they better than proxying downloads through your app?
8. How do you protect an S3 bucket against ransomware / accidental mass deletion? (versioning + MFA delete + Object Lock + replication + separate account)
9. What is S3 **request-rate** scaling and how does key prefix design relate to it?

Part 3 — Tricky

1. You grow a `gp3` volume from 100 GB to 200 GB in the console. It succeeds. `df -h` still shows 100 GB. Is AWS lying?

► **Listening for:** no — you resized the **block device**; the **filesystem** on it doesn't know. You need `growpart` (if the partition table also needs extending) and then `resize2fs` (ext4) or `xfs_growfs` (XFS). Bonus points: `lsblk` shows the new size while `df` doesn't, which is the diagnostic; XFS can only grow, never shrink; and this is why root volumes on modern AMIs run `growpart` at boot via cloud-init.

2. Your S3 bucket policy has `"Principal": "*", "Action": "s3:GetObject", "Effect": "Allow"`. Objects still return 403. Name four independent causes.

► **Listening for:** (a) **S3 Block Public Access** at bucket or account level overrides the policy — the most common answer and now on by default; (b) the object is encrypted with a **KMS CMK** and the anonymous caller has no `kms:Decrypt` (403, not a helpful error); (c) the object was uploaded by a **different account** and, with ACLs in play, the bucket owner's policy doesn't cover it (object ownership / `bucket-owner-enforced`); (d) an explicit **Deny** elsewhere — VPC endpoint policy, SCP, or a condition in the same policy (`aws:SecureTransport`, `aws:SourceIp`); (e) you're testing a signed request whose principal is evaluated differently, or requester-pays is on. Great question because "the policy says allow" is where juniors stop.

3. "S3 is eventually consistent, so after **PUT** I might read stale data." Is this still true? What actually is eventually consistent in S3 today?

► **Listening for:** since Dec 2020 S3 provides **strong read-after-write consistency** for GET/PUT/LIST/DELETE of objects, at no cost — so the classic "write a marker file and poll" workarounds in old code are obsolete. Still eventually consistent: **bucket configuration** changes (policies, ACLs, lifecycle, replication settings) and cross-region replication itself. Candidates who confidently repeat the pre-2020 model are revealing when they last learned S3 — which is a legitimate and gentle seniority probe.

4. You need a shared filesystem for 20 web servers to write user uploads. Junior proposes EFS. It works in staging and falls over in production. What went wrong, and what would you do instead?

► **Listening for:** EFS is **NFS over the network** — per-operation latency is orders of magnitude above local disk, and workloads with many small files or heavy metadata ops (`stat` storms from a web framework scanning directories, PHP/Node module resolution, `.git` operations) collapse. Add burst-credit exhaustion under sustained load, and per-GB cost $\sim 3 \times$ EBS. Better: put **uploads in S3** (that's the actual use case for object storage) and serve via CloudFront/pre-signed URLs; keep code baked in the image, not on a shared mount. Answer that names "shared filesystem is usually an architecture smell" is the senior one.

5. Multi-attach on **io2** lets you mount one volume on two instances. You put ext4 on it, mount on both, and write. What happens?

► **Listening for: corruption.** Multi-attach gives you concurrent *block-level* access; it does nothing about two independent kernels caching and writing metadata for a filesystem that assumes exclusive ownership. You need a **cluster-aware filesystem** (GFS2, OCFS2) with a distributed lock manager, or an application that does its own coordination on raw blocks. This is the same class of misunderstanding as "the LB has two instances so the DB is HA".

6. You delete 10,000 objects from a versioned bucket to save money. The bill doesn't go down. Then you can't find the objects but they're still billable. Explain, and explain how to actually clean up.

► **Listening for:** deleting in a versioned bucket writes a **delete marker** — the objects become invisible to a plain `GET / LIST` but every noncurrent version is retained and billed. To reclaim: delete specific **version IDs**, or (properly) a **lifecycle rule** on `NoncurrentVersionExpiration` plus `ExpiredObjectDeleteMarker` cleanup, and `AbortIncompleteMultipartUpload` — which is the *other* invisible cost: failed multipart uploads bill forever and appear in no object listing at all. Great real-world cost question.

7. A snapshot of a 500 GB volume completes in 20 seconds and the "first restore" of it is mysteriously slow. Explain both halves.

► **Listening for:** snapshots are **incremental** and copy-on-write against previous snapshots, and the API returns as soon as the point-in-time reference is taken — the data transfer to S3 happens in the background (so a snapshot "completing" fast is expected, and its *consistency* is another matter: you should quiesce/freeze the filesystem, or use crash-consistent + journal replay). Restore is slow because a restored volume is **lazily hydrated** from S3 on first access — blocks fault in on demand. Fix: EBS **Fast Snapshot Restore**, or pre-warm by reading the whole device (`dd if=/dev/xvdf of=/dev/null`). Anyone who knows lazy hydration has actually operated EBS.

14. Route 53 & DNS

(Lesson 15)

Part 1 — Regular

1. Walk through a DNS resolution from browser to answer. Name the resolver types (stub, recursive, root, TLD, authoritative).
2. What is a **hosted zone**? Public vs private?
3. Explain **A**, **AAAA**, **CNAME**, **MX**, **TXT**, **NS**, **SOA**.
4. What is **TTL**? Who honors it?
5. What is an **ALIAS** record in Route 53 and how does it differ from a CNAME?
6. Name Route 53's routing policies and what each is for.
7. What is a Route 53 **health check**?
8. What is a **zone apex** (naked domain)?

Part 2 — Advanced

1. Why can't you put a **CNAME at the zone apex**? What does an ALIAS record do to solve it, and why is it free?
2. Design DNS **failover** between two regions. What's the RTO, and what limits it?
3. **Weighted vs latency vs geolocation** routing — give a scenario where each is the only right answer.
4. What is **negative caching** and what governs it? Why does a typo'd record sometimes take longer to fix than a wrong one?
5. How does a **private hosted zone** get used by a VPC? What VPC settings are required?
6. Explain **DNS delegation** — you buy a domain at Registrar X and host DNS at Route 53. What exactly do you change and where?
7. What is **DNSSEC** and what attack does it prevent? Why is adoption low?
8. What is a **multivalue answer** record and why is it not a load balancer?

Part 3 — Tricky

1. You lower a record's TTL from 3600 to 60 and immediately repoint it to a new server. Hours later, some users still hit the old IP. Why, and what's the correct migration procedure?

► **Listening for:** resolvers that fetched the record *before* your change cached it with the **old 3600s TTL** — lowering the TTL only affects lookups made after it propagates. Correct procedure: lower the TTL, **wait at least one full old TTL** (ideally longer), *then* change the record, keep the old server running and serving during the drain, and raise the TTL back afterwards. Bonus: some ISP resolvers and clients (JVM, browsers, apps that cache at startup) ignore TTL entirely, so "DNS cutover" is never instant and the old endpoint should keep working or redirect.

2. **dig example.com** from your laptop returns the new IP. A user insists they get the old one. Both are telling the truth. Name four places the answer can diverge.

► **Listening for:** the browser's own DNS cache (and HTTP keep-alive/connection reuse, which skips DNS entirely), the OS stub resolver cache (`systemd-resolved`, `nsd`, `mDNS`), their **recursive resolver** (ISP / corporate / public 8.8.8.8) each with independent TTL countdowns, a corporate split-horizon or VPN DNS returning a *different zone*,

and any CDN/GeoDNS answer that legitimately varies by client location. Debug tooling: `dig @authoritative-ns` vs `dig @8.8.8.8` vs `dig` default to isolate which layer is stale. Also worth hearing: `/etc/hosts` and browser HSTS/DoH bypassing system DNS.

3. "DNS round-robin gives me load balancing and failover." Correct this.

► **Listening for:** it distributes *resolutions*, not load — no health awareness (a dead IP keeps being handed out until TTL expiry and even then it's still in the record set), no session or capacity awareness, and clients cache and reuse one answer for a long time so distribution is lumpy. As failover it's especially bad: RTO is bounded below by TTL and above by broken client caching. Route 53 **health checks + failover routing** is the real mechanism, and even then DNS is a *coarse* failover tool — for fast in-region failover you use a load balancer, not DNS.

4. Your private hosted zone `internal.example.com` resolves from your laptop over VPN but not from an EC2 instance inside the VPC. What's wrong?

► **Listening for:** the private hosted zone must be **associated with the VPC**, and the VPC needs both `enableDnsSupport` and `enableDnsHostnames`; instances must be using the VPC's **.2 resolver** (AmazonProvidedDNS) rather than a custom DNS in the DHCP option set. (The laptop-over-VPN case usually works because it's hitting a resolver you configured, or an inbound Resolver endpoint.) Extra credit: cross-account/cross-VPC resolution needs Route 53 Resolver rules or explicit authorization+association, and overlapping zone names resolve by most-specific match.

5. Someone puts a CNAME on `example.com` at a registrar that "allows" it. It seems to work, then email stops arriving. Explain.

► **Listening for:** RFC-wise a CNAME cannot coexist with **any other record at the same name**, and the apex necessarily has `SOA` and `NS` — so a CNAME there is invalid. Providers that "allow" it do CNAME flattening (fine) or actually break resolution of siblings — and the sibling that matters is **MX**: with a CNAME at the apex the MX lookup fails or is bypassed, so mail stops. This is a beautiful question because the symptom (email) is nowhere near the change (web hosting).

6. Route 53 health check marks your primary unhealthy and fails over. Your users' error rate barely improves for 5 minutes. Given TTL is 60, why?

► **Listening for:** the total failover time is **detection + propagation + client cache**: the health check needs N consecutive failed intervals (default $3 \times 30s \approx 90s$) before flipping, then the record change must propagate, then every resolver's cached 60s answer must expire, then clients with open keep-alive connections or their own caches must actually reconnect. Add that health checks come from multiple global locations requiring agreement. Realistic DNS failover RTO is minutes, not seconds — which is why critical failover happens *inside* a region at the LB layer, with DNS reserved for regional disaster.

15. IaC & Terraform Fundamentals

(Lessons 16, 17 — ClickOps vs CLI vs IaC, HCL, providers, resources, plan/apply, dependencies)

Part 1 — Regular

1. Compare **ClickOps**, the **AWS CLI**, and **IaC**. What does each give up?
2. What is **Infrastructure as Code** and what are its three main benefits?
3. **Imperative vs declarative** — which is Terraform, and what does that mean practically?
4. What is **HCL**? What is a `resource` block? A `provider` block? A `data` source?
5. What does `terraform init` / `plan` / `apply` / `destroy` each do?
6. What is a **provider**? Where does it come from and what pins its version?
7. What is `.terraform.lock.hcl` and should it be committed?
8. In `aws_subnet.foo.vpc_id = aws_vpc.bar.id` — what has Terraform learned besides a value?

Part 2 — Advanced

1. Explain Terraform's **dependency graph**. Implicit vs explicit (`depends_on`) dependencies — when do you actually need `depends_on`?
2. What is the difference between `~> 6.0`, `>= 6.0`, and `= 6.1.2` as a version constraint? Which belongs in a **module** vs a **root** module?
3. `count` vs `for_each` — when is each right? What's the classic failure mode of `count` over a list?
4. What do `create_before_destroy`, `prevent_destroy`, and `ignore_changes` do in a `lifecycle` block? Give a real use for each.
5. How does Terraform handle a resource changed outside Terraform (**drift**)? What does `terraform plan -refresh-only` show?
6. `terraform import` / `moved` blocks / `terraform state mv` — when do you reach for each?
7. What is a **provisioner** (`local-exec`, `remote-exec`) and why is it the documented last resort?
8. How do you structure Terraform for dev/staging/prod? Compare workspaces, directories, and Terragrunt-style composition.

Part 3 — Tricky

1. **FROM OUR CODE** Lesson 17 creates an instance in a VPC with:

```
resource "aws_instance" "lesson-17-instance" {
  security_groups = [aws_security_group.lesson-17-security-group.id]
}
```

Lesson 14 instead uses `vpc_security_group_ids`. One of these is a bug. Which, and what's the symptom?

► **Listening for:** `security_groups` is the **EC2-Classic / default-VPC** argument that takes security group **names**; in a non-default VPC you must use `vpc_security_group_ids` with **IDs**. Passing IDs to `security_groups` either errors or — worse — causes Terraform to see permanent drift and **force-replace** the

instance on every apply, because it keeps comparing names to IDs. Symptom: `plan` never converges, and an innocuous change destroys and recreates the instance. Lesson 14 is correct.

2. You rename a resource block from `resource "aws_vpc" "old"` to `resource "aws_vpc" "new"` and change nothing else. What does `terraform plan` show, and what do you do about it?

► **Listening for: destroy + create** — the address `aws_vpc.old` is the state key, so renaming makes Terraform believe one resource vanished and another appeared. For a VPC (or a database) this is catastrophic. Fixes: a `moved` block (declarative, reviewable, the modern answer) or `terraform state mv aws_vpc.old aws_vpc.new` before applying. Corollary they should state: **resource names are part of your infrastructure's identity**, so "just a rename" is never just a rename — and this is also why `count` indexes are dangerous.

3. ► **FROM OUR CODE** Lessons 17, 18, and 19 all contain:

```
provider "aws" {
  access_key = "AKIA..."
  secret_key = "K20q..."
}
```

You find this in a repo during a code review. What do you say, and what's the remediation order?

► **Listening for: first** — the key must be treated as **compromised the moment it's written to a file that git can see**, so remediation order is: **deactivate/delete the key in IAM first**, then rotate, then check CloudTrail for use, *then* clean the repo. Cleaning history (`git filter-repo/BFG`) does **not** un-leak it: forks, clones, CI caches, and anyone's local copy still have it, and GitHub/GitLab secret scanning may already have flagged it. Going forward: never inline credentials — use `AWS_PROFILE`/env vars, an instance role, or **OIDC federation** in CI (short-lived, no static key at all); add `pre-commit/gitLeaks` and a `.gitignore` for `*.tfvars`. The reflex to hear is "revoke before you tidy."

4. `terraform plan` says "No changes. Your infrastructure matches the configuration." You know for a fact someone changed the security group in the console an hour ago. Give two reasons `plan` can be blind.

► **Listening for:** (a) the attribute changed isn't in your configuration *and* isn't tracked as-managed the way you think — e.g. rules added via separate `aws_security_group_rule`-style resources are invisible to an inline-`ingress` SG only if that resource isn't in state, or `ignore_changes`/a missing argument means Terraform has no opinion; (b) `plan` was run with `-refresh=false` or against a **stale state** (locking off / someone else's apply not yet pushed to the backend), so it compared config to a remembered world rather than the real one. Also: the resource isn't in state at all (created outside Terraform → needs `import`), or you're pointed at the wrong workspace/backend key entirely. Diagnostic: `terraform plan -refresh-only`.

5. Two engineers run `terraform apply` against the same state at the same time. Walk me through exactly how the state file gets corrupted, and what "corrupted" means here.

► **Listening for:** both read state version N, both make real API calls, both write state — the second write **overwrites** the first, so resources that genuinely exist in AWS are absent from the surviving state. "Corrupted" doesn't mean malformed JSON; it means state no longer describes reality: orphaned resources billing forever with nothing tracking them, a later apply trying to *create* something that exists (name conflicts), or a later destroy leaving things behind. This is exactly why locking exists — see the `use_lockfile = false` question in the backends section.

6. `terraform destroy` on your dev environment. It fails partway, leaving half the infrastructure. `apply` now also fails. What's likely happening, and how do you get unstuck?

► **Listening for:** partial destroy plus dependency ordering problems — commonly a resource AWS won't delete because something *outside* Terraform depends on it (an ENI from a Lambda/LB, a non-empty S3 bucket, a `prevent_destroy`), or a resource deleted in AWS but still in state (`apply` tries to modify a ghost) or vice versa. Getting unstuck: read the actual error rather than retrying; `terraform state rm` for ghosts, `terraform import` for orphans, `-target` to make progress in slices, empty the bucket / `force_destroy`, and clear a stale lock with `force-unlock` **only** after confirming no `apply` is running. Bonus: this is the argument for `force_destroy/skip_final_snapshot` in dev and for *never* setting them in prod.

7. "Terraform is declarative, so the order I write my resources doesn't matter." Mostly true. Name two places ordering or imperativeness leaks in anyway.

► **Listening for:** (a) **provisioners** and `local-exec/null_resource/terraform_data` are imperative, run once, aren't rolled back on failure, and taint resources — they're a shell script smuggled into a declarative graph; (b) **provider configuration** must be resolvable before the resources that use it, so a provider whose credentials or region come from a resource created *in the same apply* (a classic with EKS/Kubernetes providers) fails with "provider configuration unknown at plan time" — requiring a two-phase `apply` or split state; (c) `count/for_each` values must be known at **plan** time, so anything computed from an unknown resource attribute forces `-target` or a split. Also: implicit graph edges only exist where you reference an attribute — passing a hardcoded string instead of `aws_vpc.x.id` silently deletes the ordering guarantee.

16. Terraform Variables & Modules

(Lesson 18 — variables, locals, modules, outputs)

Part 1 — Regular

1. What is a `variable` block? What can `type`, `default`, `description`, and `validation` do?
2. What is a `locals` block and how does it differ from a variable?
3. What is an `output`? Why does a module need them?
4. What is a **module**? What is the difference between a **root** module and a **child** module?
5. How do you call a local module and pass it a value? (from our `my-testing-module.tf`)
6. How do you consume a module's output in the calling module?
7. What is `path.module` vs `path.root` vs `path.cwd`?
8. Where can variable values come from? List every source.

Part 2 — Advanced

1. State the full **variable precedence** order, lowest to highest. Where do `*.auto.tfvars` and `TF_VAR_` env vars sit?
2. When should something be a `variable` vs a `local`? Give the rule you'd put in a style guide.
3. Design a reusable `vpc` module interface. What goes in as variables, what comes out as outputs, and what do you deliberately *not* expose?
4. What does `sensitive = true` actually protect, and what does it not?
5. How do you version and share modules across teams? (registry, git ref/tag source, monorepo) What's the upgrade story?
6. `for_each` over a map of objects to create N similar resources — write the pattern and explain why the map keys matter.
7. What is a **provider** inside a module? Why is passing providers into modules discouraged and how do you do it when you must (`configuration_aliases`)?
8. How do you test a module? (`terraform validate`, `plan` assertions, Terratest, `terraform test`)

Part 3 — Tricky

1. **FROM OUR CODE** Our module does:

```
output "my-vpc-lesson-18" { value = aws_vpc.my-vpc-lesson-18 }
```

and the caller uses `module.my-module-arsen.my-vpc-lesson-18.id`. It works. Why is exporting the whole resource object a bad interface?

► **Listening for:** you've published your module's **entire internal implementation** as its public API — every consumer can now depend on any attribute, so you can't refactor the resource without breaking them, and the module's contract is undocumented and unbounded. It also drags the full object (including computed and potentially sensitive attributes) into the caller's state and any `terraform output`. Better: export exactly what

consumers need (output "vpc_id" { value =id }), with descriptions. Bonus: also flags the naming — my-vpc-lesson-18 for both the resource and the output, and hyphens in identifiers, make references ambiguous to read.

2. **FROM OUR CODE** Our module declares `vpc-cidr-lesson-18` with `default = "10.0.0.0/16"`, the caller passes `"11.0.0.0/16"`, and then the caller hardcodes a subnet as `"11.0.1.0/24"`. **What's the latent bug?**

► **Listening for:** the subnet CIDR is **coupled to the VPC CIDR by convention only** — change the module input to `12.0.0.0/16` and the subnet silently becomes invalid (apply error, or worse, it lands in a different VPC's range in a copy-pasted variant). The fix is to derive it: `cidrsubnet(var.vpc_cidr, 8, 1)`, or pass subnet CIDRs as a variable validated against the VPC range. General principle worth hearing: **any hardcoded value that must agree with a variable is a bug waiting for the second engineer.**

3. You set `TF_VAR_region=eu-west-1` in your shell, `region = "us-east-1"` in `terraform.tfvars`, and run `terraform apply -var region=ap-south-1`. Which wins? Now remove the `-var`. Which wins?

► **Listening for:** `-var` wins (`ap-south-1`). Without it, `terraform.tfvars` beats `TF_VAR_` — the precedence is: `-var/-var-file` (last on the line wins) > `*.auto.tfvars` (alphabetical) > `terraform.tfvars.json` > `terraform.tfvars` > `TF_VAR_ env` > `variable default`. The counter-intuitive part people get wrong is that environment variables are **near the bottom**, not the top — so a CI job exporting `TF_VAR_*` is silently overridden by a committed tfvars file. That's a real, quiet production incident.

4. A variable is marked `sensitive = true`. Where can I still read its value?

► **Listening for:** in the **state file, in plaintext** — `sensitive` only redacts CLI/plan output. Also: it leaks through anything that consumes it (a resource attribute in state, a `local_file` on disk, an unmarked output — though Terraform errors if you output a sensitive value without marking it), through provider debug logs (`TF_LOG=DEBUG`), and through the plan file artifact if CI stores it. Therefore: state must be encrypted at rest and access-controlled like a secret store, and the real answer for secrets is to **not put them in Terraform** — reference them from Secrets Manager/SSM/Vault at runtime.

5. Your module creates 5 subnets with `count = length(var.cidrs)`. You remove the *second* CIDR from the list. What does plan show?

► **Listening for:** not one destroy — a **cascade**: `count` addresses by index, so removing index 1 shifts everything after it. Terraform plans to modify/replace subnets 1, 2, 3 and destroy 4 — potentially destroying and recreating four live subnets (and anything depending on them). `for_each` over a **map keyed by a stable identifier** addresses by key instead of position, so removing one entry destroys exactly one resource. This is the single strongest argument for `for_each`, and the reason "count is fine for identical things, for_each for anything you'll edit" is the usual rule.

6. You need one resource to exist only in prod. Someone writes `count = var.env == "prod" ? 1 : 0`. It works. Then they need it in staging too, and change the condition. What happens in prod?

► **Listening for:** nothing bad *in prod* if the resource still resolves to index 0 — but the trap is that `count`-as-a-conditional makes every reference `resource[0]`, and any change that shifts the condition's arity (or converting to a list of environments) reindexes and destroys. It also means the resource's address differs between workspaces, so

moved/import operations diverge per environment. Cleaner: `for_each = var.enabled ? toset(["this"]) : toset([])` (stable key), or separate composition per environment rather than conditionals inside a shared module. Broader point: **conditionals inside modules are how modules become unmaintainable** — prefer composition.

7. **FROM OUR CODE** Our `tmp/` folder builds a CIDR as `local.vpc-cidr-lesson-18 = "${var.testing-local-address}/${var.testing-locals-prefix}"` from two separate variables. **Critique this.**

► **Listening for:** it splits one atomic concept (a CIDR block) into two independently-settable strings with no validation, so `10.0.0.0 + 33`, or `not-an-ip + 16`, are both accepted at plan time and fail later at the API — the error surfaces far from the cause. A CIDR should be **one variable** with `type = string` and a `validation` block (`can(cidrhost(var.cidr, 0))`), and derived ranges should come from `cidrsubnet()`. General principle: variables should model the domain concept, not its string fragments, and `validation` blocks turn a runtime AWS error into a plan-time message.

17. Terraform State & Backends

(Lesson 19 — remote state, S3 backend, locking, Terraform Cloud)

Part 1 — Regular

1. What is **Terraform state** and why does Terraform need it at all?
2. What's wrong with `terraform.tfstate` on your laptop?
3. What is a **backend**? Name three.
4. What does the S3 backend need? (bucket, key, region, and historically a lock table)
5. What is **state locking** and what problem does it solve?
6. What is `terraform state list / show / rm / mv` for?
7. What is a **remote state data source** (`terraform_remote_state`)?
8. In our `terraform/` folder we use a `cloud {}` block instead. What is Terraform Cloud/HCP giving us?

Part 2 — Advanced

1. How should the S3 state bucket itself be configured? (versioning, encryption, block public access, bucket policy, and who creates it — the chicken-and-egg problem)
2. Compare S3 + DynamoDB locking with S3 native `use_lockfile` locking. What changed and why?
3. What is a **state file's** relationship to secrets? How do you protect it?
4. How do you split a large monolithic state? What are the trade-offs of many small states? (blast radius vs. cross-state coupling)
5. `terraform_remote_state` vs data sources vs SSM parameters for sharing values between stacks — rank them and justify.
6. What are **workspaces**, and why do many teams avoid them for environments?
7. Someone deleted the state file. Walk me through recovery.
8. How do you do a zero-downtime **backend migration** (local → S3, or S3 → Terraform Cloud)?

Part 3 — Tricky

1. **FROM OUR CODE** Our backend is:

```
backend "s3" {
  bucket = "terraform-state-bucket-arsen"
  key     = "state/terraform.tfstate"
  use_lockfile = false
}
```

Explain the consequence of that last line in a two-person team.

► **Listening for: no state locking at all** — concurrent applies both read state N, both mutate AWS, and the last writer wins, silently dropping the other's resources from state (orphaned, billing, untracked). It's not a warning; it's a data-loss race. Fix: `use_lockfile = true` (S3-native conditional-write locking, the modern

replacement for the DynamoDB table) or the legacy `dynamodb_table`. Extra credit: also note the backend has `no_encrypt = true`, so state — containing every attribute in plaintext — may be unencrypted, and there's no mention of bucket **versioning**, which is your only undo when state does get clobbered.

2. The state bucket must exist before the backend can use it, but you want everything in Terraform. Resolve this.

► **Listening for:** it's a genuine bootstrap problem with a few legitimate answers: create the bucket (and lock mechanism) with a tiny **separate, local-state** Terraform config committed to the repo, or with a one-off CLI/CloudFormation/script, then have everything else use it; optionally `import` it into itself afterwards. Also acceptable: use a backend that doesn't have the problem (Terraform Cloud). What matters is recognizing it's *bootstrap*, and that the bootstrap state is small, rarely changed, and can be safely versioned or even lost and re-imported.

3. State says a resource exists. AWS says it doesn't (someone deleted it in the console). What does `plan` do? Now reverse it: AWS has it, state doesn't. What does `plan` do?

► **Listening for:** case 1 — `refresh` notices it's gone and plans to **create** it (Terraform converges to config; the surprise is only if you expected an error). Case 2 — Terraform has no knowledge of it, so it plans to **create** a *second* one, which then collides on a unique name/CIDR or silently duplicates cost. That asymmetry is the point: Terraform can heal deletions but not adoptions; adoption requires `import` (or an `import` block). Follow-up: which is more dangerous in prod, and why is the answer "the one that succeeds"?

4. You run `terraform apply`, your laptop loses Wi-Fi mid-apply. What state is your state in, and what's your first command?

► **Listening for:** two problems. (a) The **lock may still be held** (stale lock) — later runs error with "state blocked"; you verify no apply is actually running, then `terraform force-unlock <id>`. (b) Resources created before the disconnect may not have been recorded — Terraform writes state incrementally and tries to persist, but a hard interruption can leave real resources untracked, and it may have written `errored.tfstate` locally. First command is **not** apply: it's `terraform plan` (after unlocking) to see reality, plus checking the console/CloudTrail for orphans and importing them. The senior instinct: read before you write.

5. Team A's stack outputs a VPC ID; Team B reads it with `terraform_remote_state`. Team A refactors and renames the output. What breaks, when, and what should the interface have been?

► **Listening for:** Team B's `plan` breaks the moment Team A applies — an invisible cross-team coupling with no versioning, no compile-time contract, and it requires Team B to have **read access to Team A's entire state** (including its secrets), which is the bigger problem. Better interfaces: publish the value to **SSM Parameter Store**/Secrets Manager (an explicit, versionable, access-scoped contract), or have Team B look the VPC up by **tag with a data source** (loosely coupled, no state access), or make the shared network a module both consume. Ranking these correctly is a strong senior signal.

6. Is `terraform state rm` a way to delete a resource?

► **Listening for:** **No — it's the opposite.** `state rm` makes Terraform *forget* the resource; the resource keeps existing and keeps billing, now untracked and unmanaged (a deliberate orphan). `terraform destroy`/removing the config is how you delete. `state rm` is for legitimate surgery: moving a resource to another state, un-managing something intentionally, or clearing a ghost. The modern declarative version is a `removed` block. Anyone who reaches for `state rm` to fix errors is about to create orphans.

7. You migrate from the S3 backend to Terraform Cloud. `terraform init` offers to copy state, you say yes, everything works. Name two things that are now different that nobody warned you about.

► **Listening for:** (a) **where apply runs** — HCP/Terraform Cloud default is *remote* execution, so the run happens on their workers: your local env vars, `AWS_PROFILE`, SSO session, `~/.aws/credentials`, and any `local-exec` assumptions are gone; credentials must be configured as workspace variables (or OIDC/dynamic credentials), and local files referenced by `path.module` must be in the uploaded config. (b) **state access, locking, and versioning move to their control plane** — locking is now automatic and queued (good), state history is retained on their side (compliance question: where does that data live?), and the old S3 copy is a stale duplicate someone will eventually mistake for authoritative — so it should be locked down or deleted deliberately. Also: run ordering/queueing changes CI assumptions, and `terraform apply` may now require a UI approval.

18. Kubernetes Architecture

(Lesson 20 — what problems k8s solves, control plane vs nodes, pods, etcd, kubelet, CNI, dockershim)

Part 1 — Regular

1. What does "k8s" stand for and why the 8?
2. What problems does Kubernetes solve? (from our list: single point of failure, resource limits, horizontal + vertical scaling, restart-on-release, feature rollout)
3. What is a **cluster**? What is a **node**?
4. **Control plane (master) vs worker node** — name the components on each and what they do.
5. What is a **Pod**? Can a Pod hold more than one container? Who gets the IP — the Pod or the container?
6. What is **etcd** and what's stored in it?
7. What does the **kubelet** do? What does **kube-proxy** do?
8. Name two ways to connect to a cluster. (kubectl, Lens) What does **kubeconfig** contain?

Part 2 — Advanced

1. Trace `kubectl apply -f deploy.yaml` end to end: which components touch it, in what order, and where does the decision to place a Pod on a node happen?
2. What is the **CNI** and what does a network plugin have to provide? What is the flat-network requirement ("every Pod can reach every Pod")?
3. Compare **flannel** and **Calico**. When do you need Calico specifically?
4. Why do you back up **etcd** and the **certificates**? What does a restore actually restore?
5. How many control-plane nodes should a production cluster have and why? What is the etcd quorum math?
6. What is a **sidecar** container and what does it share with the main container? Give two real patterns.
7. **Why did Kubernetes "reject Docker"**? Be precise about what was removed and what still works.
8. What are **requests** vs **limits**, and how does the scheduler use each?

Part 3 — Tricky

1. **FROM OUR CODE** Our Lesson 20 notes list "flannel, kaniko, canal" together. **Which of flannel, Calico, Canal, and Kaniko is not a CNI plugin, and what is it actually?**

► **Listening for:** Kaniko is a tool for building **container images inside a cluster without a Docker daemon** (no privileged mode, no socket mount) — it belongs to the CI/build conversation, not networking. Flannel (simple overlay, no network policy), Calico (L3 with NetworkPolicy support and BGP), and Canal (flannel networking + Calico policy) are the CNI options. The bonus insight: our note "if you don't want any network policy, use flannel" is right for the wrong-sounding reason — flannel doesn't *support* NetworkPolicy, so "no policy" isn't a choice you make, it's a capability you lack.

2. "Kubernetes dropped Docker, so my Docker images won't run." Correct this fully.

► **Listening for:** what was removed in 1.24 is **dockershim** — the kubelet's adapter for talking to the Docker daemon, which was maintenance burden once CRI existed. Images are unaffected because they're OCI images (Docker builds OCI images; containerd/CRI-O run them), and Docker itself uses containerd underneath. What *does* break: anything depending on the Docker daemon on nodes — `docker ps` for debugging, Docker-in-Docker CI patterns that mount `/var/run/docker.sock`, node-level logging agents reading Docker's log format, and `docker build` on a node. Replacements: `crictl`, Kaniko/BuildKit for builds. This is a great question because the panic-headline version and the true version are very different.

3. A Pod has two containers. Container A listens on `localhost:8080`. Can container B reach it on `localhost:8080`? Can a container in a *different* Pod on the same node?

► **Listening for:** yes for B — containers in a Pod share the **network namespace** (one IP, one port space, shared loopback), which is also why two containers in a Pod **cannot both bind 8080**. No for the other Pod: it has its own netns and must go via the Pod IP or a Service. Follow-ups worth asking: what else is shared by default (IPC optional, UTS, and volumes you mount into both) and what isn't (filesystem, PID namespace unless `shareProcessNamespace: true`)?

4. Your 3-node control plane: one node dies. Cluster fine? Two die. What exactly stops working — and can existing Pods still serve traffic?

► **Listening for:** 1 of 3 down → quorum is 2, still met, cluster fully functional. 2 of 3 down → **no etcd quorum**, so the API server can't accept writes: no new deployments, no scheduling, no self-healing, and controllers can't act. **But** already-running Pods keep running and kube-proxy keeps forwarding, so user traffic often continues — the cluster is *unmanageable*, not *down*. That distinction (control plane vs data plane) is exactly what separates people who've operated a cluster. Follow-up: why does adding a 4th control-plane node not help?

5. You restore an etcd snapshot from 3 days ago onto a running cluster. What happens?

► **Listening for:** the API's view of the world reverts, but **the real world didn't** — Pods created since then are still running on nodes but absent from etcd (so controllers see missing replicas and create duplicates, or garbage-collect them), resources deleted since then reappear and get recreated, and Secrets/ServiceAccount tokens and certificate state may no longer match what nodes hold. That's why restore is a **whole-cluster, offline procedure** (stop API servers, restore on all etcd members, matching certificates and CA — hence "back up etcd *and* the certs"), not a live rollback. Also: a snapshot without its CA is nearly useless, which is the real reason our lesson lists both.

6. A Pod is **Pending** forever. `kubectl describe` shows no scheduling error you recognize. Give five independent causes.

► **Listening for:** insufficient allocatable CPU/memory for its **requests** (the scheduler uses requests, not limits, and not current usage); **taints** without matching tolerations (including control-plane and `NotReady`/pressure taints); `nodeSelector`/affinity/anti-affinity or topology-spread constraints no node satisfies; an unbound **PVC** (no matching PV / wrong storage class / zone mismatch with the node); node count zero or all nodes `NotReady` / cluster-autoscaler unable to add capacity; `ResourceQuota` or `LimitRange` rejecting it; image-pull issues are `ImagePullBackOff` not `Pending` (worth catching the distinction). The senior move is naming `kubectl describe pod` events and `kubectl get events --sort-by` as the actual diagnostic path.

7. "Kubernetes fixes single points of failure." Your cluster has 3 control-plane nodes, 10 workers, and one HA app. Name three single points of failure that remain.

► **Listening for:** the **cluster itself** (one cluster = one failure domain: a bad upgrade, an etcd corruption, a CNI or admission-webhook misconfiguration, a cert expiry takes everything); the **stateful dependency** — one database Pod, one PV in one AZ, one EBS volume that's zonal; **shared infrastructure** — the single ingress/LB, the container registry (can't pull = can't recover), DNS (CoreDNS outage breaks all service discovery), the cloud region; and the **human/config layer** — one Helm chart, one CI pipeline, one admin credential. Extra credit: an admission webhook with `failurePolicy: Fail` and no running backend can wedge the entire API. The right instinct is "k8s relocates SPOFs, it doesn't abolish them."

19. Kubernetes Manifests

(Lesson 21 — Pod, Deployment, Service, labels/selectors, NodePort, resources)

Part 1 — Regular

1. What are the four required top-level fields of any manifest? (`apiVersion`, `kind`, `metadata`, `spec`)
2. What is a **Deployment** and what does it give you over a bare Pod?
3. What is a **ReplicaSet** and who creates it?
4. What are **labels** and **selectors**? How does a Service find its Pods?
5. Explain a Service's `port`, `targetPort`, and `nodePort`.
6. Compare `ClusterIP`, `NodePort`, and `LoadBalancer`.
7. What are `resources.requests` and `resources.limits`? What does `100m` CPU mean? `128Mi`?
8. `kubectl apply` vs `kubectl create` vs `kubectl replace`?

Part 2 — Advanced

1. What is an **Endpoints** / `EndpointSlice` object and how does it relate to a Service?
2. Explain the **QoS classes** (Guaranteed, Burstable, BestEffort). How are they assigned and what do they control?
3. How does a **rolling update** work? What do `maxSurge` and `maxUnavailable` do, and what's the interaction with **readiness probes**?
4. Compare **liveness**, **readiness**, and **startup** probes. What happens when you get liveness wrong?
5. Why is a Deployment's `selector.matchLabels` **immutable**, and what do you do when you must change it?
6. What is an **Ingress** and why would you use it instead of a `LoadBalancer` Service per app?
7. `ConfigMap` and `Secret` — how do you consume them, and what's the difference between env-var and volume mounting on update?
8. How would you achieve zero-downtime deploys? List every piece that must be right.

Part 3 — Tricky

1. **FROM OUR CODE** Our `my-pod.yaml` has the Service active with `selector: {imonq-en: hahaimnen}` while **both** the Deployment and the Pod are commented out. `kubectl apply -f my-pod.yaml` succeeds. Then `curl <node-ip>:30080` hangs. Walk me through the diagnosis.

► **Listening for:** the Service exists and is perfectly valid — Services don't validate that anything matches their selector, they just resolve to an (empty) **EndpointSlice**. Diagnosis path: `kubectl get endpoints nginx-service` / `kubectl get endpointslices` → **no endpoints** → the selector matches zero Pods → because no workload was applied. The key insight: **a Service with no endpoints is not an error, it's a silent black hole** — this is the #1 real-world K8s debugging pattern, and the label-selector indirection is the thing to check first (`kubectl get pods --show-labels`). Also worth noting: `kubectl apply -f` of a file where everything else is commented out is a great way to silently delete-by-omission if you're using `apply --prune`.

2. **FROM OUR CODE** Our commented-out Pod requests `cpu: 100m / memory: 128Mi` with limits `cpu: 200m / memory: 256Mi`. What QoS class is that Pod, and under node memory pressure, in what order does the kubelet evict?

► **Listening for: Burstable** — because limits are set but don't *equal* requests (Guaranteed requires requests == limits for every container and both resources; BestEffort means nothing set). Eviction order: **BestEffort first, then Burstable** (ranked by usage above requests), **Guaranteed last**. So the practical lever is: set requests == limits for anything you don't want evicted. Follow-up gold: which of these two limits gets you throttled and which gets you killed? (CPU 200m → CFS throttling, silent latency; memory 256Mi → OOMKilled, exit 137) — which ties straight back to the Lesson 5 overcommit question.

3. **FROM OUR CODE** Our Service is `type: NodePort` with `nodePort: 30080`. You apply it in EKS, the Pods are running and healthy, endpoints are populated, and `curl <public-node-ip>:30080` still times out. Two causes.

► **Listening for:** (a) the node's **security group** doesn't allow inbound 30080 from your IP — NodePort opens a port on every node's *kernel*, and nothing tells AWS about it; (b) you're hitting a node's **private** IP, or the node has no public IP, or it's in a private subnet. Also: `externalTrafficPolicy: Local` means only nodes actually running a Pod respond (the others blackhole), and the valid nodePort range is **30000–32767** so a value outside it is rejected at apply time. Broader point they should reach: NodePort is a debugging/edge primitive — production uses an Ingress or a `LoadBalancer` Service so the cloud LB and SG are managed for you.

4. You change a Deployment's `selector.matchLabels`. `kubectl apply` fails. You `kubectl delete deployment --cascade=orphan` and reapply with the new selector. What's the state of your cluster now?

► **Listening for:** you have **two generations of Pods running simultaneously** — the orphaned old ReplicaSet's Pods (now owned by nothing, never reconciled, never cleaned up) plus the new Deployment's Pods. If a Service selector matches both, traffic splits across two versions unpredictably; if it matches neither, nothing serves. And nothing will ever remove the orphans but you. (`--cascade=orphan` is occasionally the right tool for a live migration, but only with a plan to reconcile.) The correct move for a selector change is a **new Deployment name** with a controlled traffic cutover, since the selector is the ReplicaSet's identity.

5. A liveness probe hits `/health`, which checks the database connection. The database has a 30-second blip. Describe what happens to your 20 Pods.

► **Listening for:** every Pod fails liveness simultaneously → the kubelet **restarts all 20** → they all come up cold, all reconnect at once (thundering herd on an already-struggling DB), possibly failing again → `CrashLoopBackOff` across the fleet, turning a 30-second dependency blip into a multi-minute self-inflicted outage. The rule: **liveness = "is this process wedged and only a restart fixes it"** (deliberately shallow, dependency-free); **readiness = "can I serve right now"** (may check dependencies, and only removes you from endpoints — reversible, no restart). Getting this backwards is one of the most common and most damaging K8s mistakes.

6. `kubectl apply` a manifest that omits a field you previously set. The field reverts. Now `kubectl edit` a field, then `apply` the original file again — your edit is silently kept. Why the difference?

► **Listening for:** `apply` computes a three-way merge using the `kubectl.kubernetes.io/last-applied-configuration` annotation (or Server-Side Apply's field ownership): a field you *previously applied* and then removed from the file is deleted, but a field you never applied — set by `edit`, by a mutating webhook, by HPA, or

by a defaulter — isn't owned by you, so `apply` leaves it. Consequences: mixing `edit/scale/patch` with `apply` produces state that no file describes (and an HPA fighting a hardcoded `replicas:` in your manifest is the classic version of this), which is the whole argument for GitOps and Server-Side Apply with explicit field managers.

7. You scale a Deployment to 3 replicas. All 3 land on the same node. Nothing is misconfigured. Why, and how do you spread them?

► **Listening for:** the default scheduler optimizes for a **valid, reasonably-scored** placement, not for spreading — with plenty of capacity on one node, `LeastAllocated`-style scoring plus a small cluster can pile all three there; the Deployment controller has no spreading concept at all. Consequence: one node failure takes down 100% of a "3-replica HA" service. Fixes: `topologySpreadConstraints` (over `kubernetes.io/hostname` and `topology.kubernetes.io/zone`), pod **anti-affinity**, and a **PodDisruptionBudget** so voluntary disruptions (drains, upgrades) can't take them all at once. Naming PDB unprompted is a strong signal.

20. Cross-cutting / Scenario Round

Use these as 10–20 minute whiteboard or debugging exercises. They deliberately span several lessons, which is where real seniority shows.

Part 1 — Regular (design something small)

1. Design the deployment for a Node app + Postgres for a 3-person startup. Draw it. Justify every box on cost grounds.
2. A developer says "it works on my machine". Give five things you'd check, ordered.
3. Walk me from `git push` to "the new version is serving traffic" in this repo's stack (GitLab CI → registry → EC2/K8s). Name every artifact along the way.
4. Our repo has AWS keys committed in three Terraform files. Write the incident checklist.
5. You inherit an environment with no documentation. What are the first five commands you run, in each of: a Linux host, a Docker host, an AWS account, a K8s cluster?

Part 2 — Advanced (trade-offs and failure)

1. The same app must run in dev (docker compose), staging, and prod (K8s). What is shared and what necessarily differs? Where does config live?
2. Design monitoring for the three-tier app from Lesson 8. What are the four golden signals per tier, and what would you actually page on?
3. Your Terraform state is in S3, your team is now 12 people across 4 squads. Restructure. Justify the state boundaries.
4. Cost review: a \$40k/month AWS bill. Name the eight line items you check first and the specific waste each usually hides.
5. Write the runbook for "the site is down" for the Lesson 11 architecture (VPC, bastion, private instances behind an LB, NAT). Assume you're paged at 3am.

Part 3 — Tricky (multi-layer, no clean answer)

1. A deploy goes out at 14:00. At 14:40, p99 latency triples but error rate is flat and CPU is normal. The deploy diff is one line: a new `HEALTHCHECK` on the container and a new readiness probe path. Explain a plausible causal chain.

► **Listening for:** the shape of the reasoning, not one answer. Candidate chains: the probe endpoint is expensive (touches the DB) and now runs every 10s × N replicas, adding sustained load and connection-pool contention → queuing → p99 up with no errors; or readiness now flaps under load, so Pods leave and rejoin endpoints, shrinking effective capacity while the LB reshuffles connections; or the healthcheck process itself is heavy enough to trigger CPU **throttling** inside the cgroup, which doesn't show as high host CPU (ties to Lesson 5). What you're grading: do they separate *saturation* from *errors*, do they suspect the thing that changed, and do they know throttling is invisible in the obvious metric.

2. "We have 99.99% availability" — the app is on 3 replicas across 3 AZs behind an ALB, with an RDS Multi-AZ, all in Terraform, all monitored. Find the availability lie.

► **Listening for:** the redundant compute isn't the constraint. Real limits: RDS Multi-AZ failover is **60–120 seconds** of write unavailability (that alone is more than 99.99%'s ~4.4 min/month budget if it happens twice), it's one *region*; schema migrations, certificate expiry, DNS TTL, the container **registry**, the CI pipeline, and CoreDNS/Route

53 are all shared single dependencies; nothing above covers **bad deploys**, which are the actual leading cause of downtime, and 99.99% requires progressive delivery + fast rollback, not more replicas. The strongest answers question whether availability is even measured from the **user's** perspective (synthetic checks against the LB $\neq$ user success rate) and whether there's an error budget at all.

3. Someone reports "the deploy is stuck". Give me the decision tree that distinguishes: image pull failure, readiness probe failure, insufficient cluster capacity, a stuck Terraform lock, a CI runner queue, and an unapproved manual gate — using at most 5 commands total.

► **Listening for:** a triage instinct that goes **outside-in** and cheap-to-expensive: is the *pipeline* even running (CI UI / job status — catches runner queue + manual gate); did the deploy command return (Terraform lock error / `kubectl rollout status`); then `kubectl get pods` (one glance separates `ImagePullBackOff` / `Pending` / `Running` but `0/1 Ready` / `CrashLoopBackOff`); then `kubectl describe pod` events for the why (pull auth vs capacity vs taint); then `kubectl logs` for app-level readiness failure. Bonus for knowing `kubectl get events --sort-by=.lastTimestamp` and that "stuck" plus healthy old Pods means the rollout is *correctly refusing* to proceed — which is a feature, and the old version is still serving.

4. Your company wants to cut the AWS bill 30% in a quarter. Your infra is Lesson 11–14 shaped: VPC with NAT gateways, ASG of EC2, EBS, EFS, S3, an RDS. Where do you actually look, and what will your first instinct get wrong?

► **Listening for:** the big items are usually **not** the ones people optimize. Look at: NAT gateway per-GB processing (huge for chatty S3/ECR traffic → **gateway VPC endpoints** eliminate it), **cross-AZ data transfer** (invisible and often enormous), idle/unattached **EBS volumes and old snapshots**, over-provisioned instance sizes (rightsizing beats reserved pricing on the wrong size), **Savings Plans/RIs** on the *stable* baseline only, S3 lifecycle + incomplete multipart uploads + noncurrent versions, EFS in the wrong throughput mode, RDS running prod-sized in dev and dev environments running 24/7 for a 40-hour work week. The wrong instinct: buying reservations first (locks in current waste), or chasing a cheaper instance family before measuring — and the meta-point: **you cannot cut what you can't attribute**, so cost allocation tags come before optimization.

5. Pick any three tools from this course and argue we should *not* be using them. Then rebut yourself.

► **Listening for:** genuine engineering judgment rather than tool loyalty. Strong candidates argue things like: Kubernetes is enormous accidental complexity for a team that could run ECS/Fargate or three VMs; Terraform's state is a liability and CDK/Pulumi or a managed platform avoids it; Ansible overlaps with immutable images and should be replaced by Packer + golden AMIs; Docker Compose in prod is a trap; NAT gateways are a tax you can architect away. The rebuttal matters more than the critique — the answer to grade highly is one that identifies *at what scale or with what constraints* each choice flips, and admits that "boring and well-understood by this team" is a legitimate technical argument.

Appendix — Interviewer's quick guide

Signals that matter more than the answer:

- Distinguishing *timeout* vs *refused* vs *denied* when debugging (network vs process vs auth).
- Knowing the difference between the **control plane** and the **data plane** of anything.
- Saying "revoke first, tidy later" about a leaked credential.
- Noticing when a question's premise is wrong and saying so.
- Answering "it depends" **and then naming what it depends on**.
- Reaching for "what changed?" before "what's broken?".

Red flags:

- "Cloud is cheaper."
- "The tests pass, so it works."
- Confidently reciting pre-2020 S3 consistency, or "unhealthy containers auto-restart".
- Reaching for `terraform state rm` or `allow_failure: true` to make an error go away.
- Never mentioning cost, teardown, or blast radius unprompted.

A note on the ► questions: all of them come from real code in this repo. They're the best questions in the bank because the code *looks* correct — which is exactly the condition under which real bugs ship.